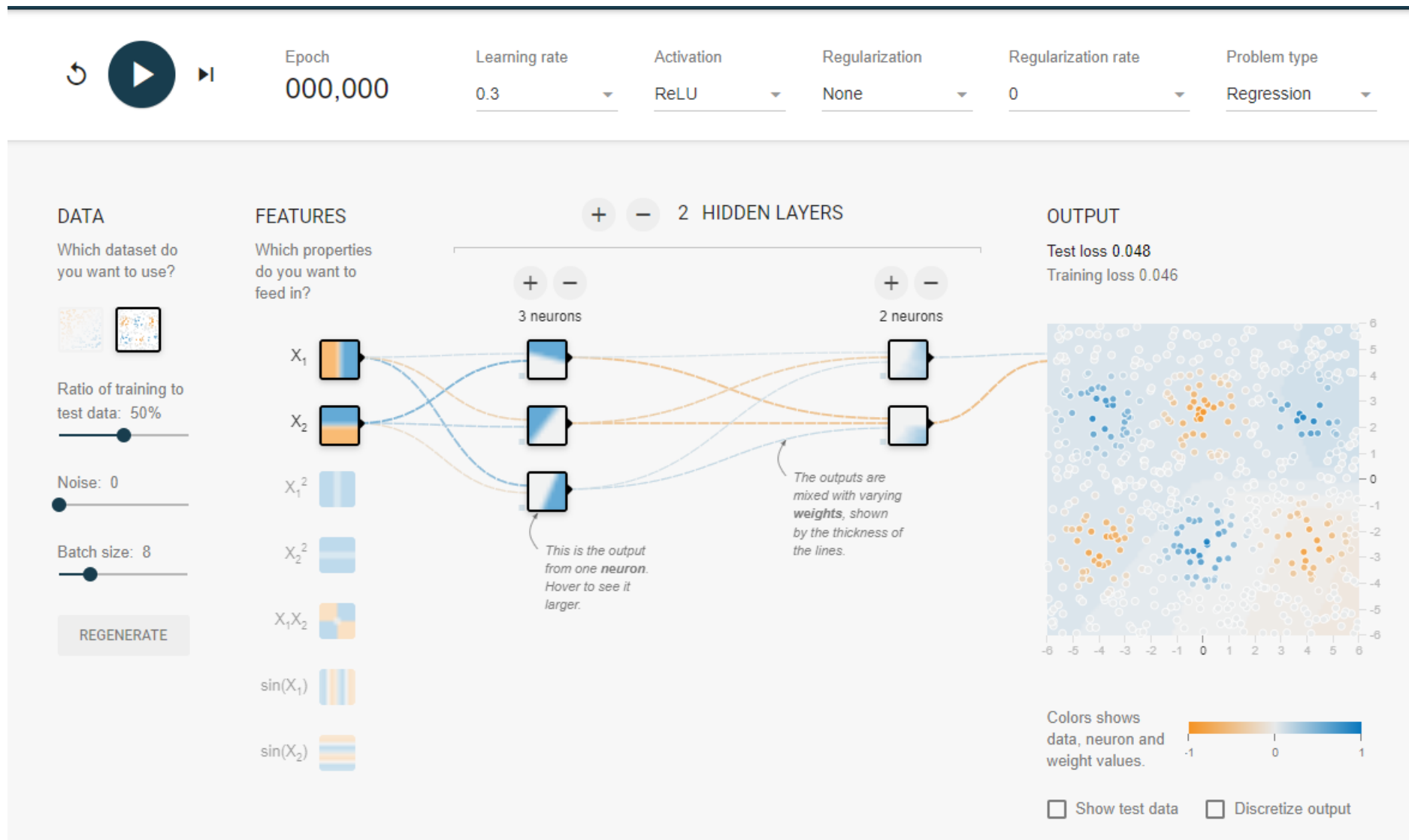


Review

텐서플로우 플레이 그라운드: <https://playground.tensorflow.org/>



**“딥러닝이 어떻게 작동하는지,
눈으로 직접 보고 실험해보는 곳!”**

“사용자가 브라우저에서 바로 신경망 실험”

“교육용 시각화 툴”

“신경망 놀이터

💡 누가 쓰면 좋아요?

- **딥러닝 초보자:** 이론보다 먼저 "이렇게 동작하는구나!"를 감으로 익힐 수 있어요
- **선생님/교육자:** 직관적으로 가르치기 딱 좋은 시각자료
- **실험 좋아하는 사람:** 하이퍼파라미터 조절의 효과를 직접 실험 가능

검색: 텐서프로 플레이그라운드



텐서프로 플레이그라운드



전체

이미지

동영상

쇼핑

뉴스

⋮ 더보기

도구

검색결과 약 2,220개 (0.46초)



TensorFlow

<https://playground.tensorflow.org> ⋮

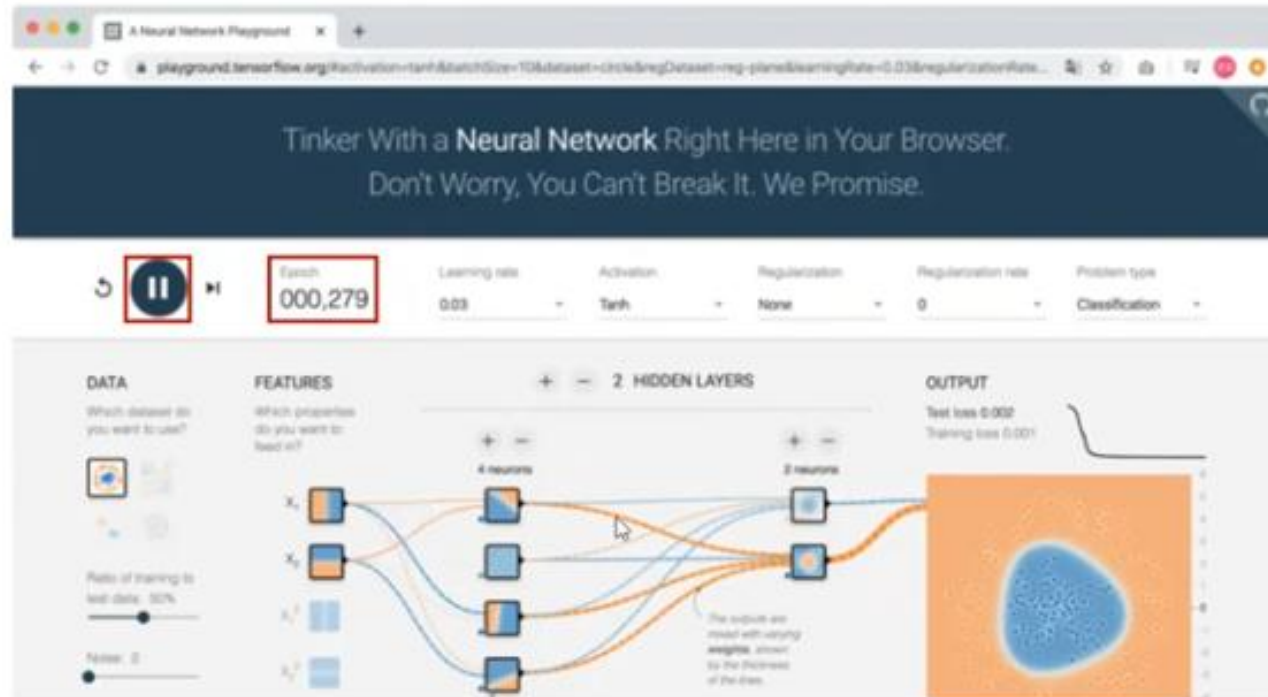
A Neural Network Playground - TensorFlow

We wrote a tiny neural network library that meets the demands of this educational visualization.

For real-world applications, consider the **TensorFlow** library.

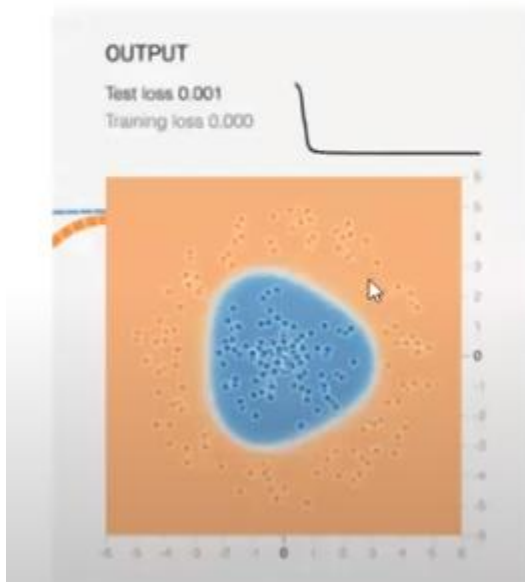
1. 시작 버튼

- 시작 버튼(▶)을 누르면 애니메이션 효과가 나오며, 이런 저런 값들이 변하는 모습을 살펴볼 수 있다
- 먼저 화면 왼쪽 위의 에포크(Epoch)라고 적힌 숫자의 값이 늘어나는 것을 볼 수 있다
- 에포크란 전체 데이터를 사용하여 인공 신경망이 학습한 횟수를 의미



2. 출력부분

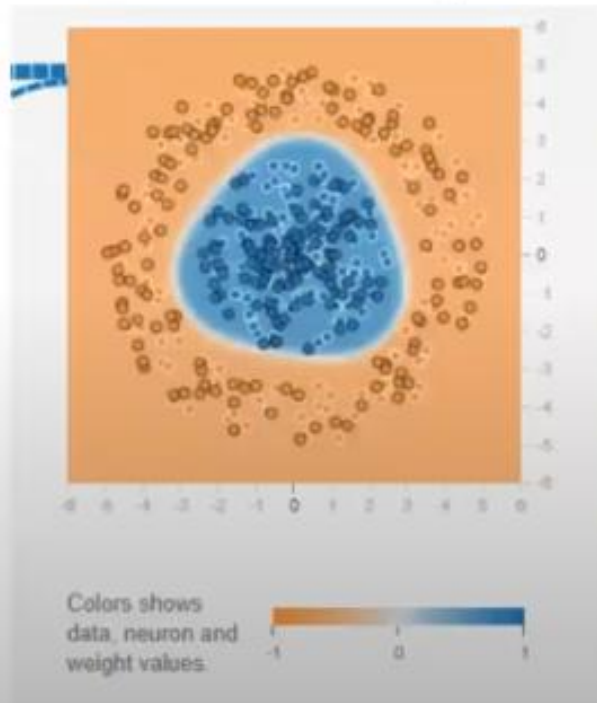
- 에포크가 늘어나면서 변하는 모습 중 하나는 바로 출력(OUTPUT)으로, 그래프와 데이터 영역이 변하는 것을 볼 수 있다



- 파란색 점이 있는 곳과 주황색 점이 있는 곳이 2개 영역으로 구분됨
- 그래프에서 볼 수 있듯이 오차 또한 0에 가까워지고 있다

2. 출력부분

- 텐서플로 플레이그라운드의 데이터는 훈련 데이터(training data)와 검증 데이터(test data)로 구분되어 있다
- 화면에서 Show test data 버튼을 클릭하면 어떤 것이 검증 데이터인지 살펴볼 수 있다

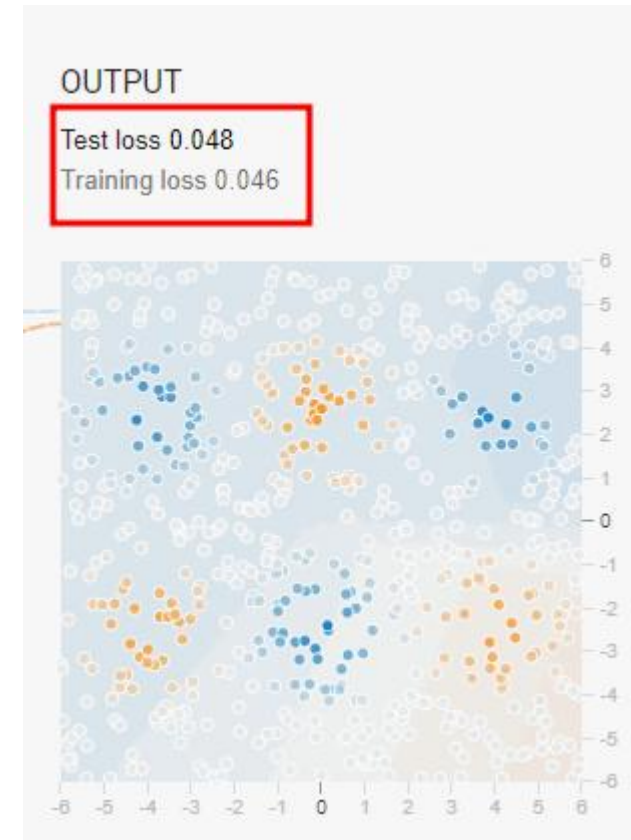


- 데이터를 훈련 데이터와 검증 데이터로 구분하는 이유는 신경망을 훈련하는 데 모든 데이터를 쓰지 않고 일부분만을 사용하기 위해서이다
- Test loss는 검증 데이터를 신경망에 넣었을 때 오차값을 의미한다
- Training loss는 훈련 데이터에 대한 오차값을 말한다
- 학습이 점점 진행되며 각각의 오차값이 줄어드는 것을 볼 수 있고 이 값의 변화를 그래프로도 살펴볼 수 있다

“출력의 Test *Loss, Training loss값이
0에 가까울 수록 학습이 잘 일어난 것
입니다.”

*Loss(손실값) = 예측값 - 실제값

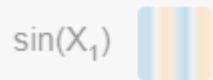
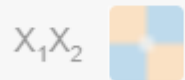
“신경망이 얼마나 잘못 예측했는가? 지표”



신경망에 어떤 입력정보(특성, 변수)를 넣을까?

FEATURES

Which properties
do you want to
feed in?



Feature 이름

뜻

설명

X_1

x좌표

점의 가로 위치 (수평 방향 정보)

X_2

y좌표

점의 세로 위치 (수직 방향 정보)

X_1^2

x좌표 제곱

곡선(비선형) 경계를 잡는 데 유용

X_2^2

y좌표 제곱

마찬가지로 곡선 경계를 위해 사용

🎯 왜 다양한 Feature를 선택하나요?

- x_1 , x_2 만 사용하면 선형적인 경계만 학습하기 쉬움
- x_1^2 , x_2^2 를 추가하면 → 신경망이 곡선, 원형, 비선형 패턴도 잘 학습 가능
- 복잡한 데이터셋일수록 다양한 Feature가 필요해요

“컴퓨터가 점을 구분하려면, 각 점의 위치 정보를 숫자로 받아야 해요. 그 숫자를 바로 'Feature'라고 해요.”

“단순한 위치 말고, 제곱 값 같은 추가 정보가 있으면 더 복잡한 구분도 가능해져요!”

고양이 VS 강아지 분류 문제 예시

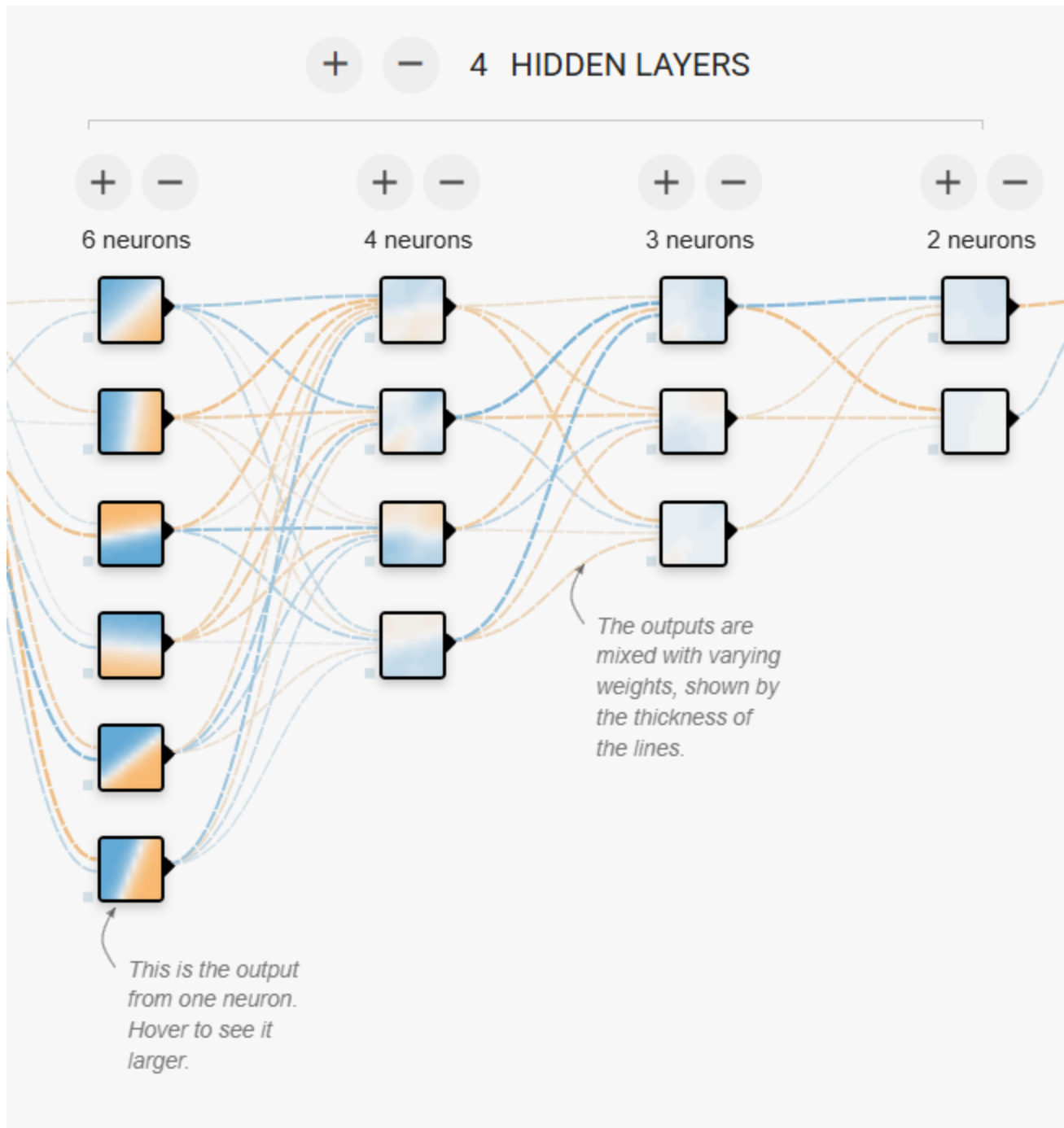
항목	의미	고양이 vs 강아지에서의 비유
x_1	첫 번째 입력 특성	예: 귀의 길이
x_2	두 번째 입력 특성	예: 눈의 크기
x_1^2	첫 번째 특성 제곱	귀가 아주 긴 경우 강조
x_2^2	두 번째 특성 제곱	눈이 매우 큰 경우 강조
x_1x_2	특성 간 곱 (상호작용)	귀와 눈이 동시에 클 때 효과 강함
$\sin(x_1)$	첫 번째 특성의 사인값	주기적 변화 표현 (예: 줄무늬, 반복무늬)
$\sin(x_2)$	두 번째 특성의 사인값	예: 진동성 있는 특징 감지 (예: 털 무늬 주기)

Sin(x) 값의 의미

1. 주기 함수
2. 예) 고양이의 줄무늬가 일정한 주기로 반복된다면 SIN 함수로 표현해야 특징을 추출할 수 있다.
3. 모델이 직선이 아닌 **곡선, 반복** 패턴등을 찾아낼 수 있도록 해 줌
4. 복잡한 패턴이 있는 데이터일수록 이런 특성들이 더 큰 힘을 발휘한다.

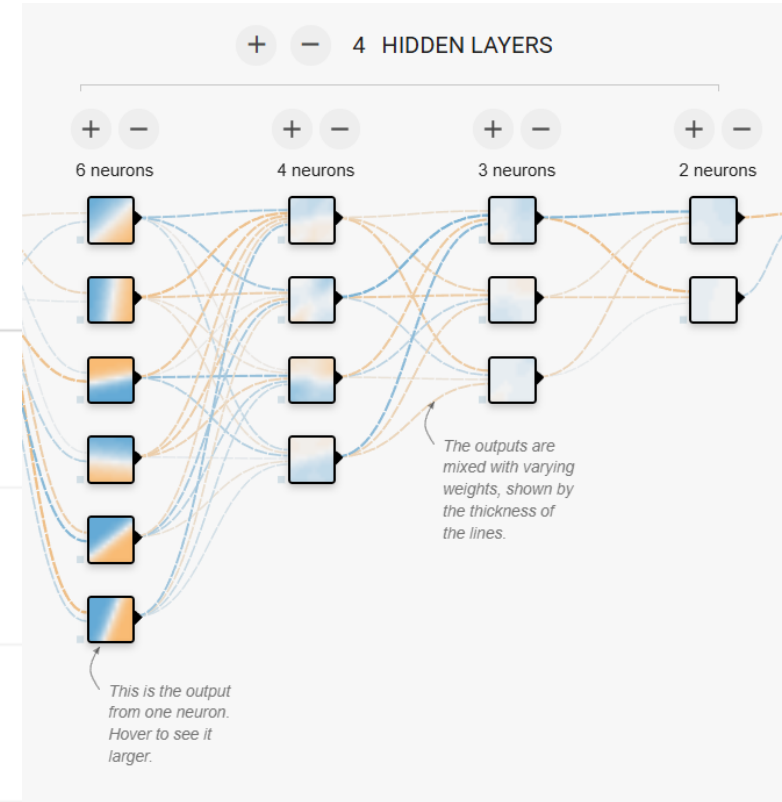
히든레이어와 노드

- 히든 레이어(Hidden Layers): 총 4개 층
- 각 레이어당 뉴런 수:
 - 1층: 6개
 - 2층: 4개
 - 3층: 3개
 - 4층: 2개



히든레이어 & 고양이 강아지 분류

레이어	역할	예시
1층 (6 뉴런)	기본 특징 감지	귀 길이, 눈 크기, 코 모양 등 개별 정보 분석
2층 (4 뉴런)	특징 조합 시작	귀가 크고 눈이 작으면 강아지 가능성 ↑
3층 (3 뉴런)	고차원 패턴 탐색	고양이 품종 특유의 얼굴 구조 인식
4층 (2 뉴런)	최종 판단 전 통합	"지금까지 종합해보면 고양이일 확률이 높다"



텐스플로우 플레이그라운드
'신경망 관련 용어'

손실함수~

정답과 얼마나 다른냐?

모델확률분포와 실제 확률분포가 얼마나 다른가?

딥러닝 학습목표

- 손실함수 최소화
- 가중치(weight)와 편향(bias) 조정

손실함수 종류

1. MSE(Mean Squared Error)
2. RMSE(Root Mean Squared Error)
3. MAE(Mean Absolute Error)
4. Binary Cross-entropy
5. Categorical Cross-entropy

“자세한 것은 나중에 ~~”

손실함수(Loss function)란?

1. 모델의 예측값과 실제 정답 사이의 차이를 수치화해주는 함수
 - AI 모델이 얼마나 잘 예측했는지를 **정량적으로 평가**
 - 손실함수 최소화: 모델 학습의 핵심 목적

#예시(날씨 예보 모델)

실제기온(20도) // 모델 예측(17도) // 차이 3도

 손실값 계산 (예: 평균제곱오차, MSE)

$$\text{Loss} = (20 - 17)^2 = 9$$

- 이 **9**라는 숫자가 바로 손실함수가 계산한 **예측 오류의 크기**예요.
- 이 값이 **0에 가까울수록** 모델이 정확하게 예측한 것이고,
- **값이 클수록** 예측이 많이 틀렸다는 뜻입니다.

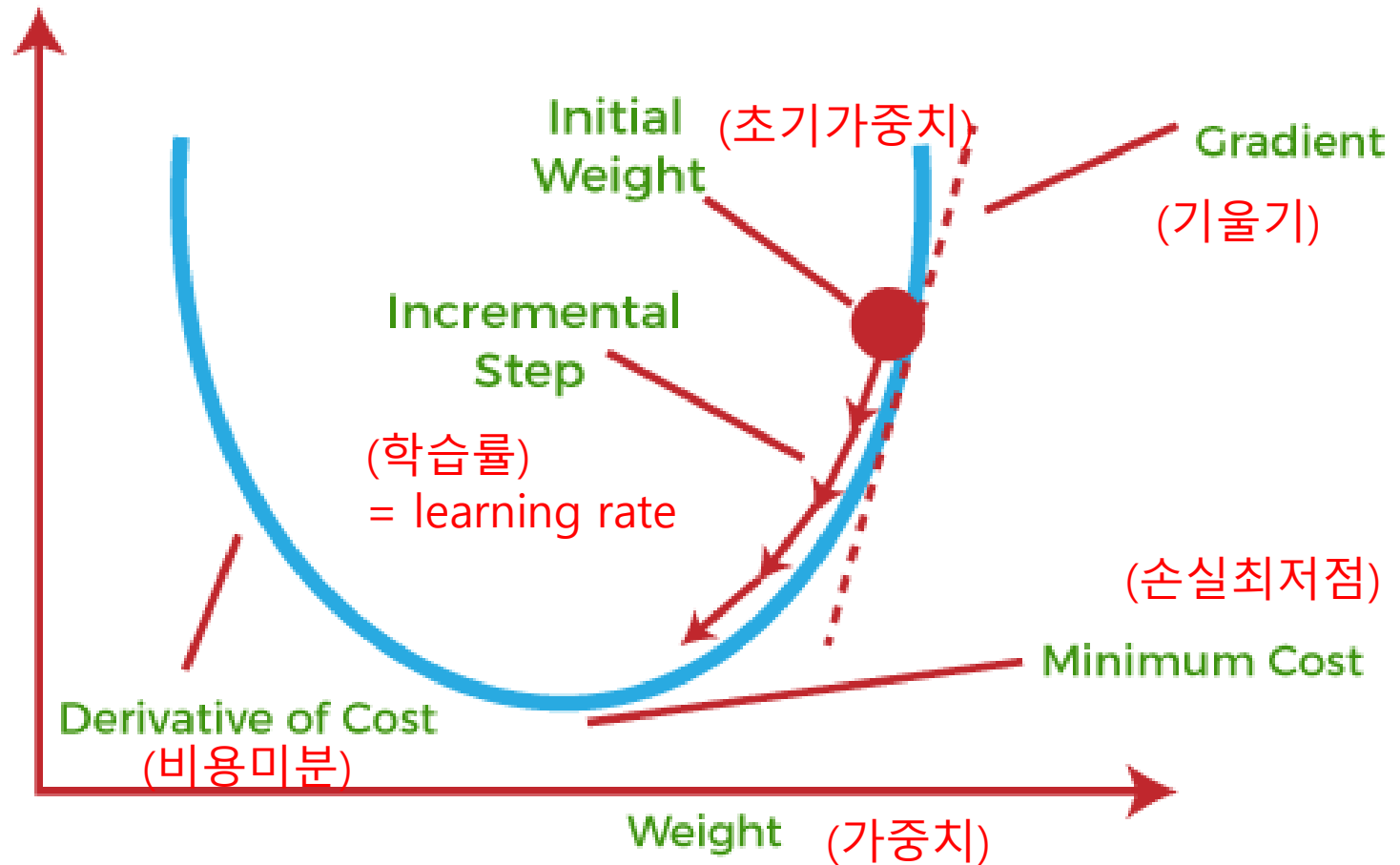
손실함수(Loss function)와 딥러닝 학습

1. 모델의 예측값과 실제 정답 사이의 차이를 수치화해주는 함수
 - AI 모델이 얼마나 잘 예측했는지를 **정량적으로 평가**
 - 손실함수 최소화: 모델 학습의 핵심 목적
2. 손실함수는 그래프로 표현이 가능하다.
 - 평균제곱오차 (MSE)는 곡선 형태의 **2차 함수로 그려짐**
3. **그래프의 한 지점에서 미분이 가능하다.**
 - 수학적으로 그 함수가 **연속적이고, 매끄럽다**는 뜻
4. 미분결과를 바탕으로 손실 값이 최소가 되는 방향으로 모델을 학습시킬 수 있다.

옵션 1. Learning rate(학습률)

- 딥러닝의 핵심 개념인 경사 하강법과 관련한 용어
- 정답값과 예측 값의 오차를 최소화하는 방식으로 학습 진행
- 오차값을 최소화하기 위해 각 뉴런을 연결하는 가중치 값을 수정해 나가는데 한 번 수정할 때 얼마만큼 수정할까? 학습률로 결정

Gradient descent(경사하강)



경사하강법이란?

“ 주어진 손실함수에서 모델의 파라미터의 최적의 값을 찾는 머신러닝과 딥러닝의 최적화 알고리즘의 하나이다”

* 파라미터: 연결가중치 또는 편향

2. Activation(활성화 함수)

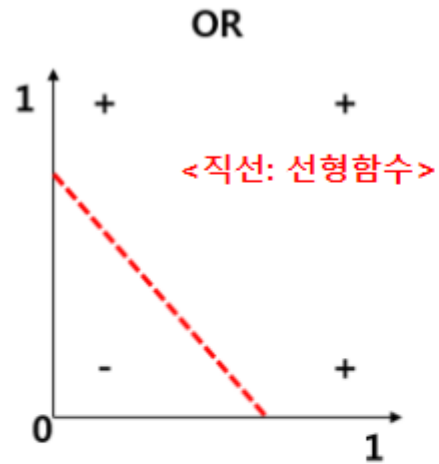
1. 입력신호를 비선형적으로 변환해 출력하는 함수
2. 선형 대 비선형
 - 선형: 직선그래프(기울기 일정)
 - 비선형: 곡선, 꺾임, 굴곡(기울기 일정하지 않음)
3. 비선형으로 변환하는 이유
 - 미분을 통해 손실함수 최소화 방향 찾기 위해
 - 복잡한 패턴을 학습할 수 있음
4. ReLU, tanh, Sigmoid, Linear 함수를 사용

AI가 “뉴런이 이 신호가 중요한가를 판단해서
반응할까? 말까?” 판단하는 스위치”

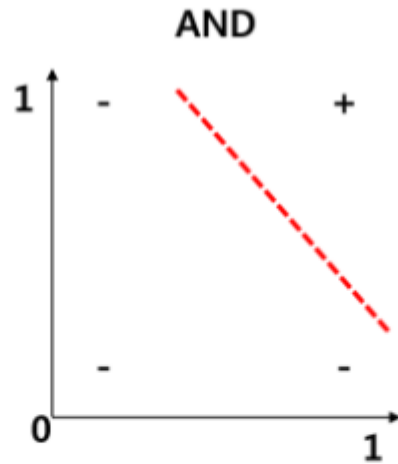
활성화 함수 = 스위치

- 스위치(활성화 함수)가 없다면, AI는 아무리 계산해도 계속 **직선 계산만 반복**해서 **복잡한 문제**는 못 푼다.
- 신호가 약하면 스위치 = OFF
- 신호가 강하면 스위치 = ON

인공지능의 암흑기 (XOR 해결까지 30년이 걸림~)



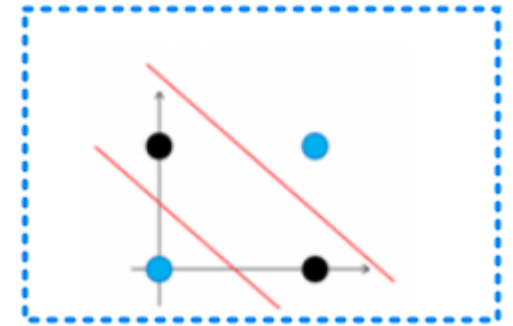
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1



x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



“두 직선=히든레이어의
신경망의 노드 개수”

◆ 1. 인공지능 개념 등장 – 1956년

- 다트머스 회의(Dartmouth Conference)
- 존 매카시(John McCarthy), 마빈 민스키(Marvin Minsky) 등:
"기계를 사람처럼 생각하게 만들 수 있다"는 **AI의 시작 선언**

◆ 2. 퍼셉트론(Perceptron) 등장 – 1958년

- 프랭크 로젠블랫(Frank Rosenblatt)이 만든 **단층 퍼셉트론 모델**
- 선형 분류 문제는 잘 풀 수 있었지만, **XOR 문제는 절대 못 풀**
- 퍼셉트론이 할 수 있는 건, 선 하나로 나눌 수 있는 문제뿐 (예: AND, OR)

🔥 3. XOR의 벽 – 1969년

- 마빈 민스키 & 시모어 페퍼트의 책 《Perceptrons》 출간
→ 단층 퍼셉트론은 XOR 같은 **비선형** 문제를 풀 수 없다는 사실을 수학적으로 증명

📉 이로 인해 AI 연구에 대한 기대가 꺾였고,

👉 약 10년간 AI 연구가 침체됨 (AI 겨울, **First AI Winter**)

“**히든 레이어 1개 이상이면 다중퍼셉트론**”

● 4. 다층 퍼셉트론 (MLP)와 역전파(Backpropagation)의 부활 – 1986년

- 제프리 힌튼(Geoffrey Hinton) 외 연구팀이
역전파 알고리즘을 이용한 **다층 퍼셉트론(Multi-Layer Perceptron)** 학습 방법을 제시
- 이 구조는 은닉층을 추가하여 XOR 문제도 해결 가능!



개와 고양이 사진 분류 문제

① 📷 입력 (Input)

- 개나 고양이 사진을 숫자(픽셀 값)로 바꿔서 AI에게 보여줘요.
예: 사진 한 장 → `[0.2, 0.9, 0.4, ...]` (숫자 수천 개)

② 🧱 히든 레이어 여러 개

- AI는 이 숫자들을 계속 계산하면서,
사진 속에서 중요한 특징들(귀, 수염, 눈 모양 등)을 하나씩 추출해요.
- 층이 많을수록:

- 첫 번째 층: 기본적인 선과 밝기 찾기
- 두 번째 층: 눈, 귀, 수염 등 부위 구분
- 세 번째 층: 전체 얼굴 모양이나 포즈 파악

③ 활성화 함수의 역할 (ReLU 등)

각 히든층에서 AI가

“이 특징, 중요한가?”를 판단하는 **스위치 역할**을 해요.

- 예: ReLU는 0보다 큰 신호만 통과시켜요 → **중요한 정보만 다음 층으로 전달**

만약 이걸 안 쓰면, AI는 선만 쪽쪽 긋는 바보가 돼요 😊

→ 복잡한 패턴을 **절대** 못 배워요!

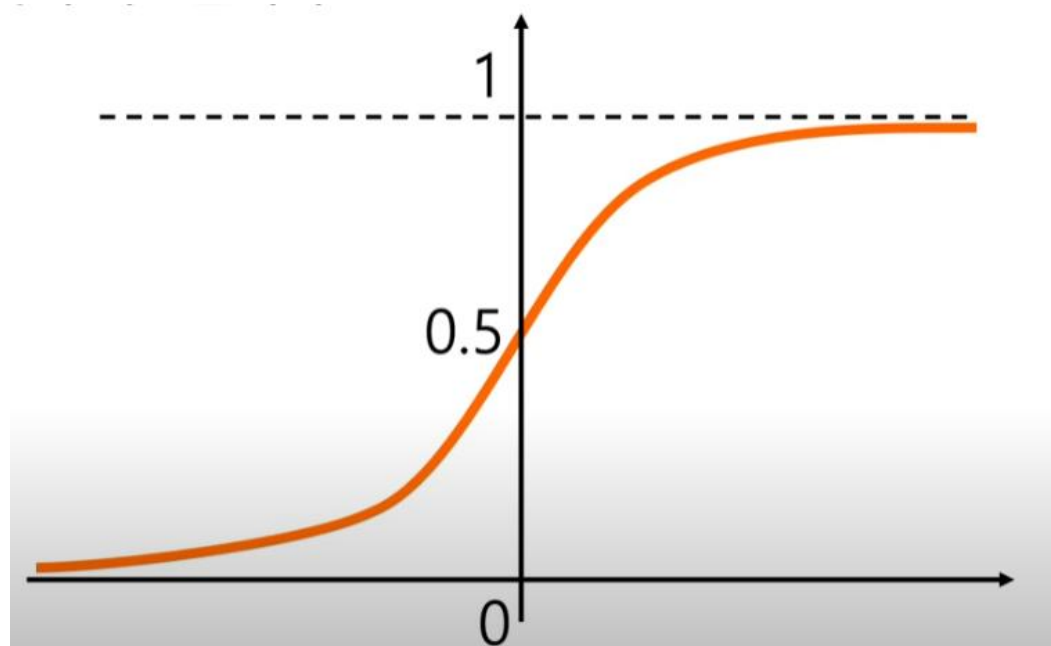
④ 출력

- AI가 “개일 확률: 0.2, 고양이일 확률: 0.8”

→ **고양이!** 라고 예측해요.

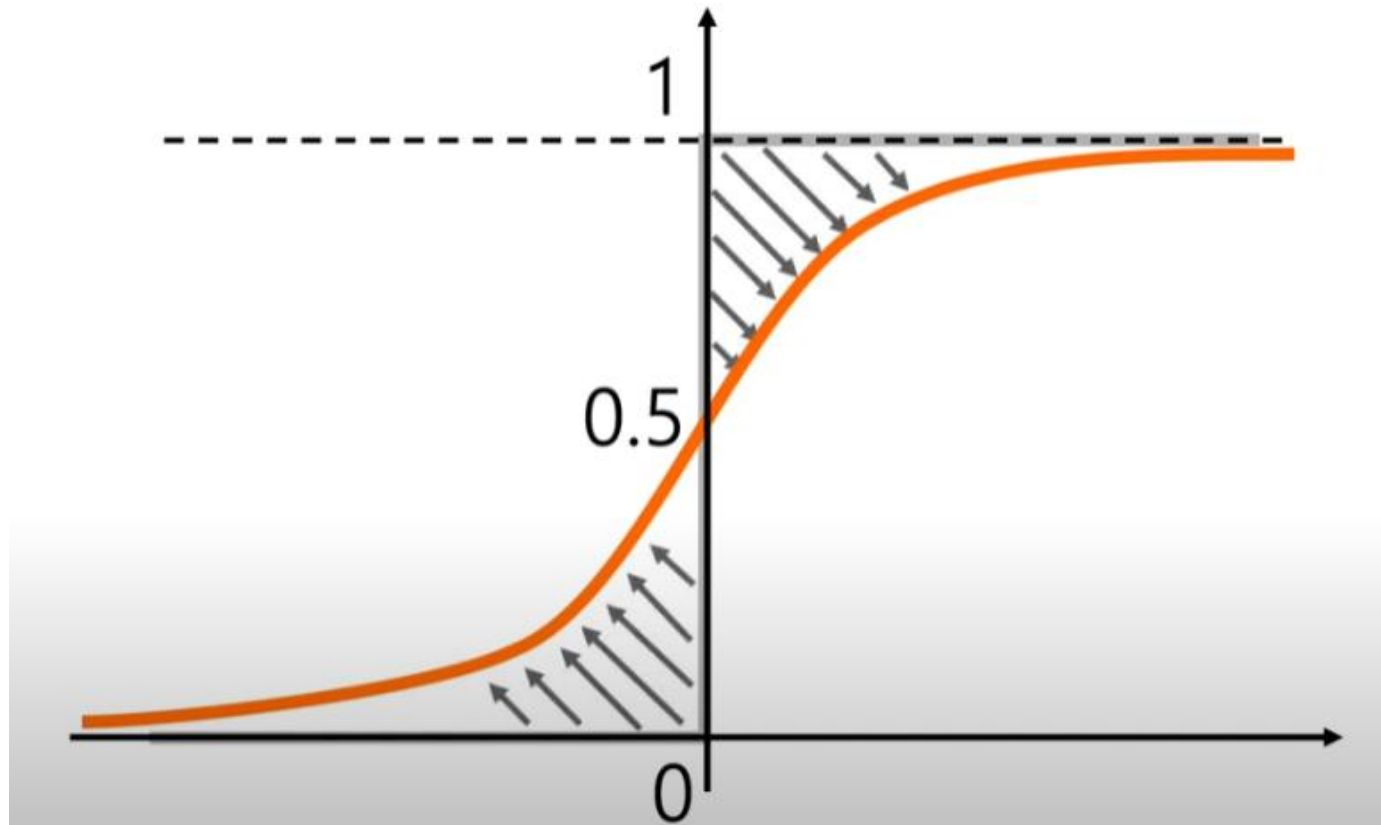
1. 시그모이드 활성화 함수

1. 인공신경망의 핵심 구성요소
2. 신경망의 비선형 문제를 해결할 수 있는 중요한 역할
3. 활성화 함수가 없다면...신경망 층을 깊게 쌓는 것이 의미가 없다



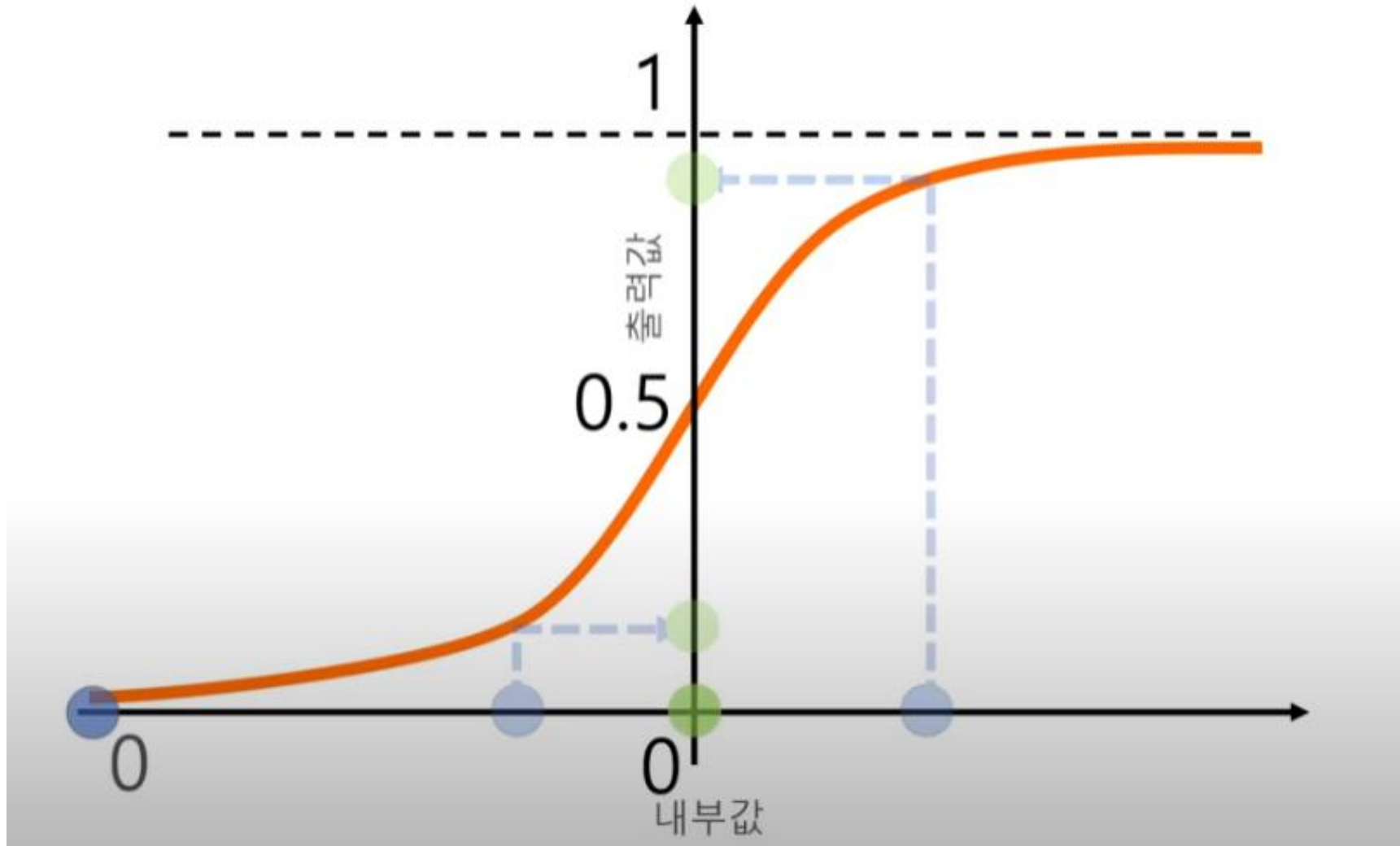
S자형의 대표적 함수 '시그모이드 함수'

"미분 가능한 함수"



"부드러운 차이 다층신경망 시대의 혁신을 열다"

0과 1로만 출력값을 갖는 계단 함수에 비해...훨씬 다양한 오차를 계산할 수 있다.

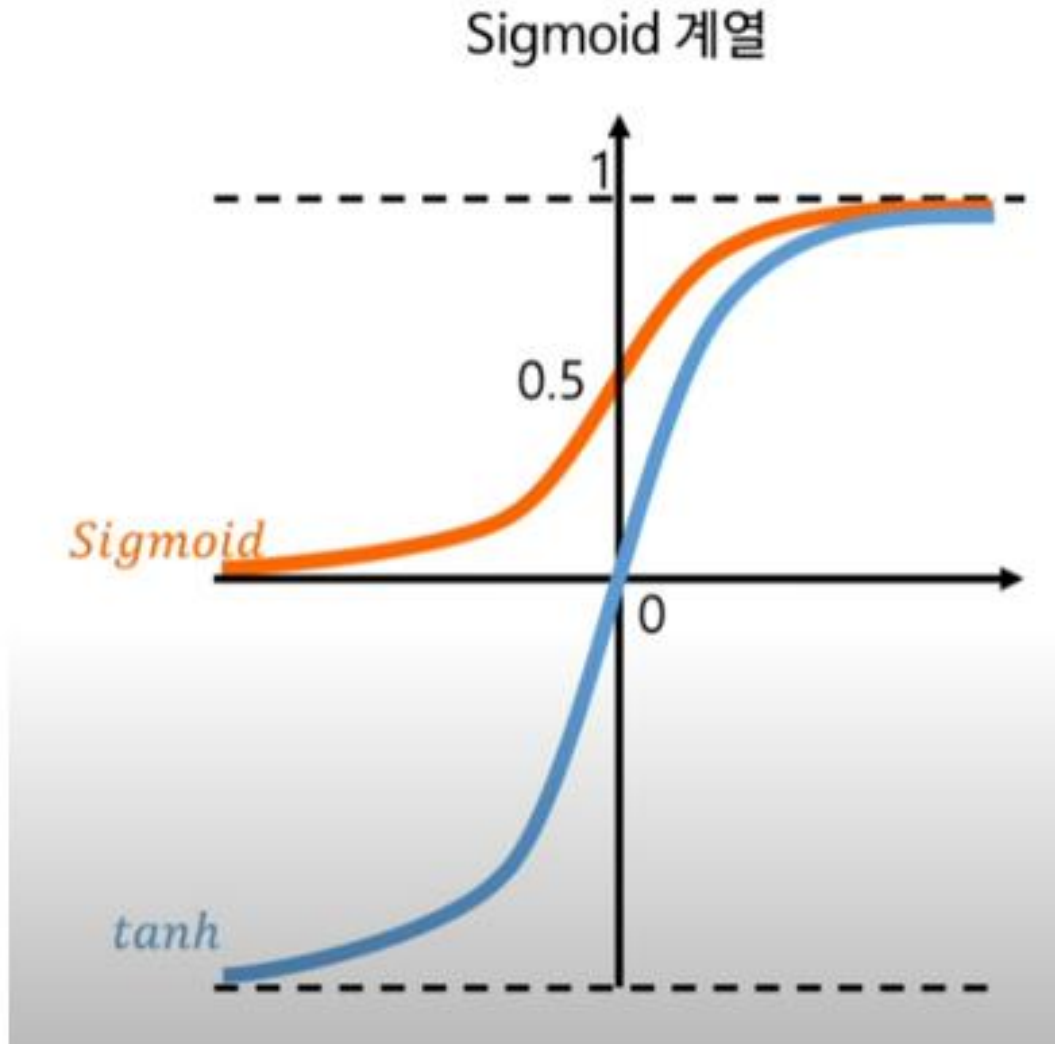


- 입력값 : -무한대에서 +무한대까지
- 출력값 : 0에서 1

<하이퍼볼릭 탄젠트 함수>

"Non Zero-centered 문제"

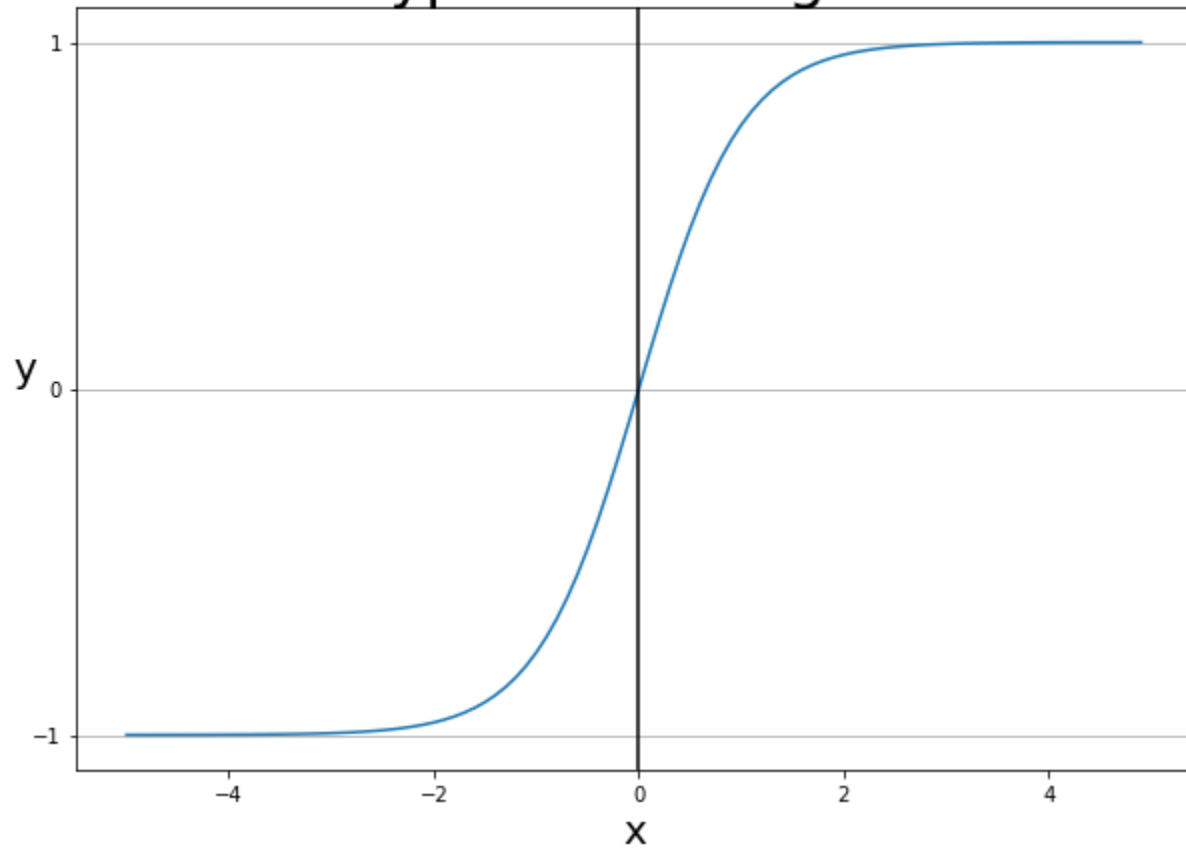
- 0이 중심이 아니다
- 출력값의 비대칭 문제 발생
- 학습의 효율성 저하
- 기울기 업데이트 비효율



"출력값 -1에서 +1"

"하이퍼볼릭탄젠트함수는 Non Zero-centered 문제해결 "

Hyperbolic Tangent



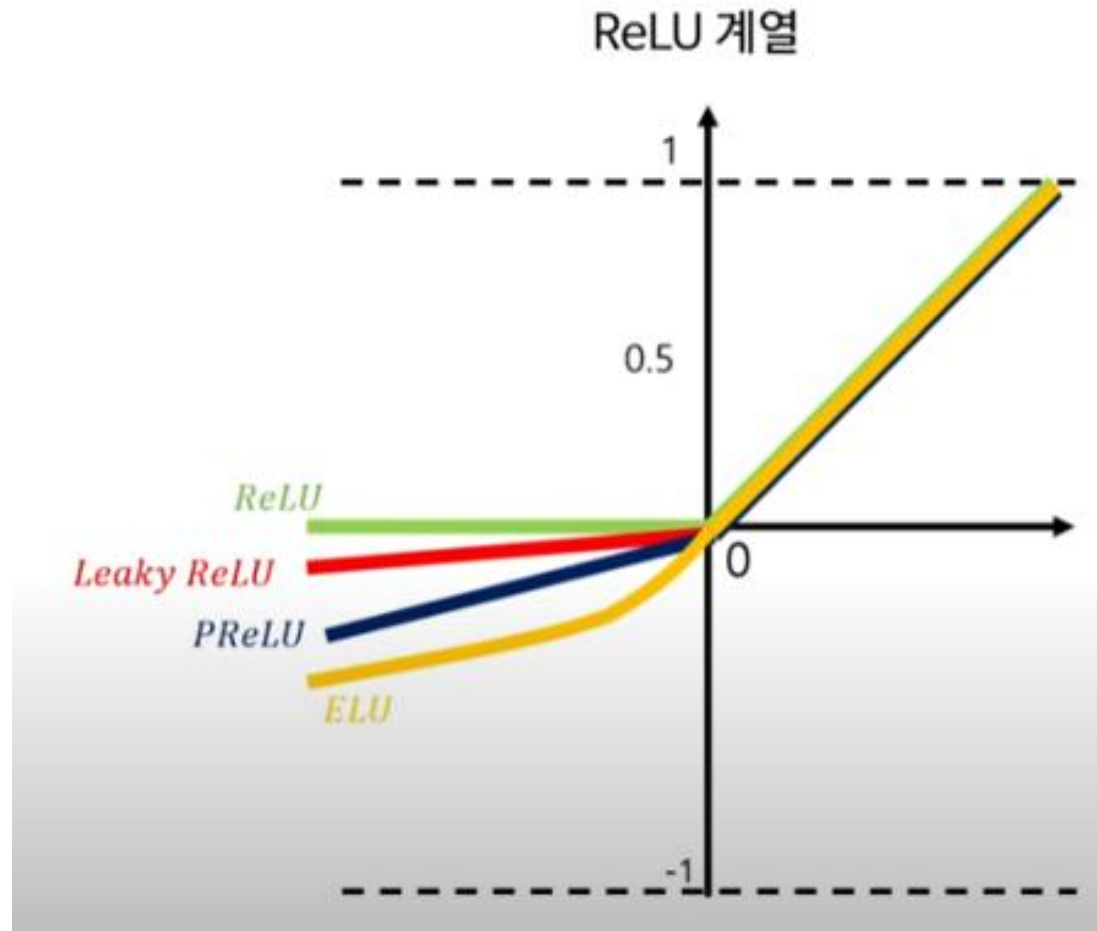
"미분(1) = 기울기(1)=정비례
입력=출력 "

	시그모이드 함수	하이퍼볼릭 탄젠트 함수
범위	0 ~ 1	-1 ~ 1
중앙값	0.5	0
미분 최대값	0.3	1

<시그모이드 함수 대안>

“하이퍼볼릭 탄젠트함수도 Non-zero 문제 해결 못함”

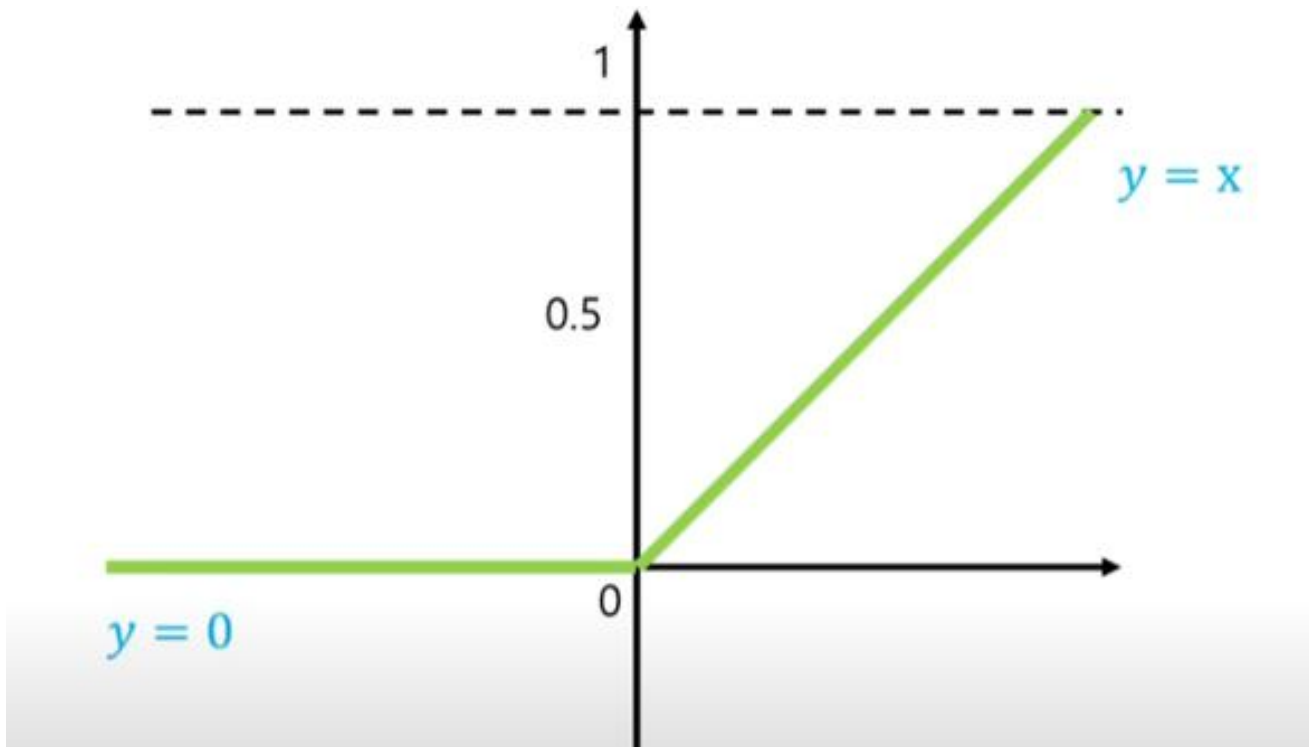
그래서...ReLU 함수



RuLU 함수

“Vanishing gradient problem(기울기 소실문제) 해결”

“Deep Learning 혁신”

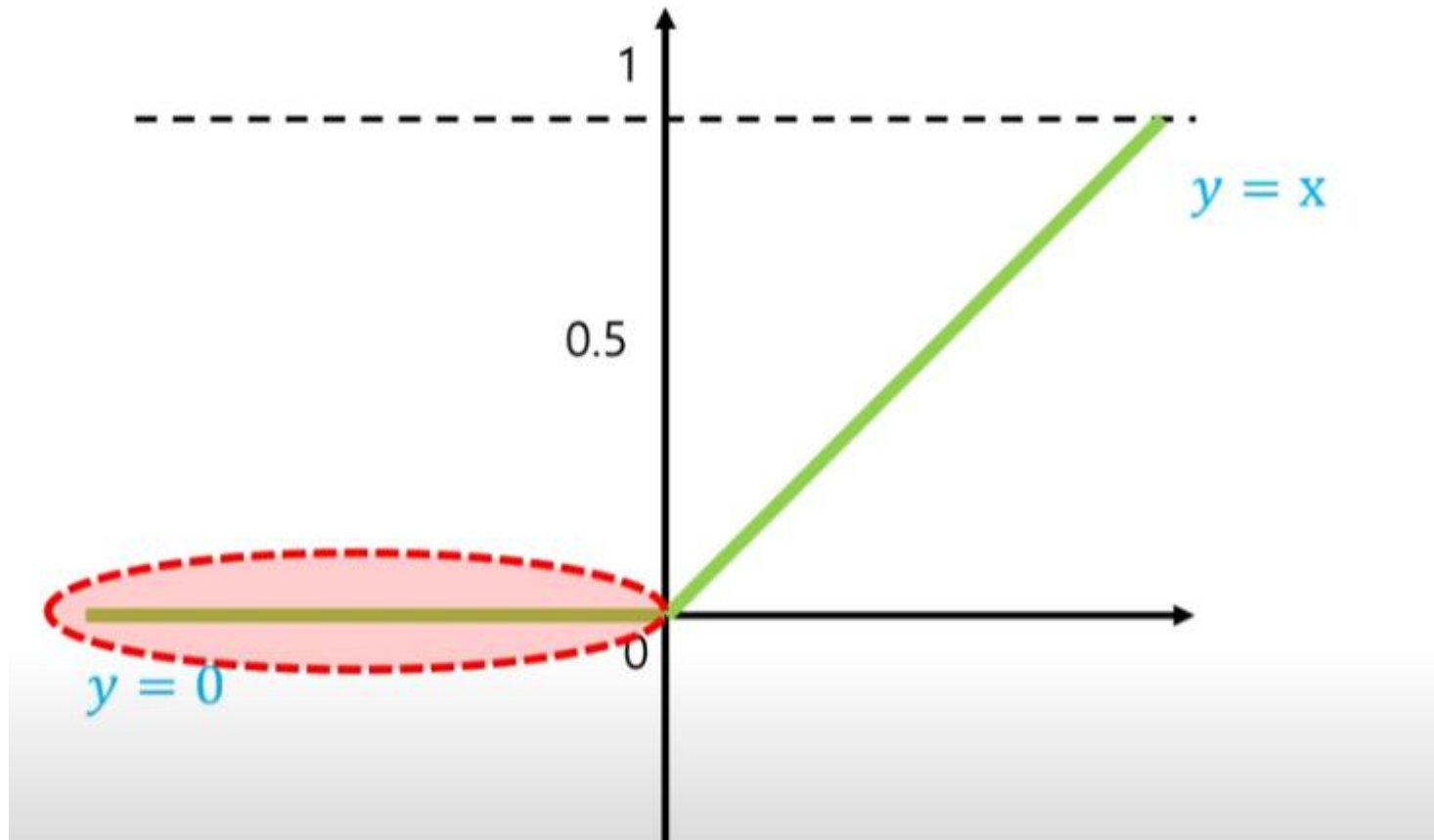


“(양수) 함수의 기울기 1로 고정
기울기 소실문제 발생 안함”

“신경망의 깊은 층까지 오류역전파를 통한 학습 가능함”

ReLU 함수의 약점 '입력값 음수=기울기 0'

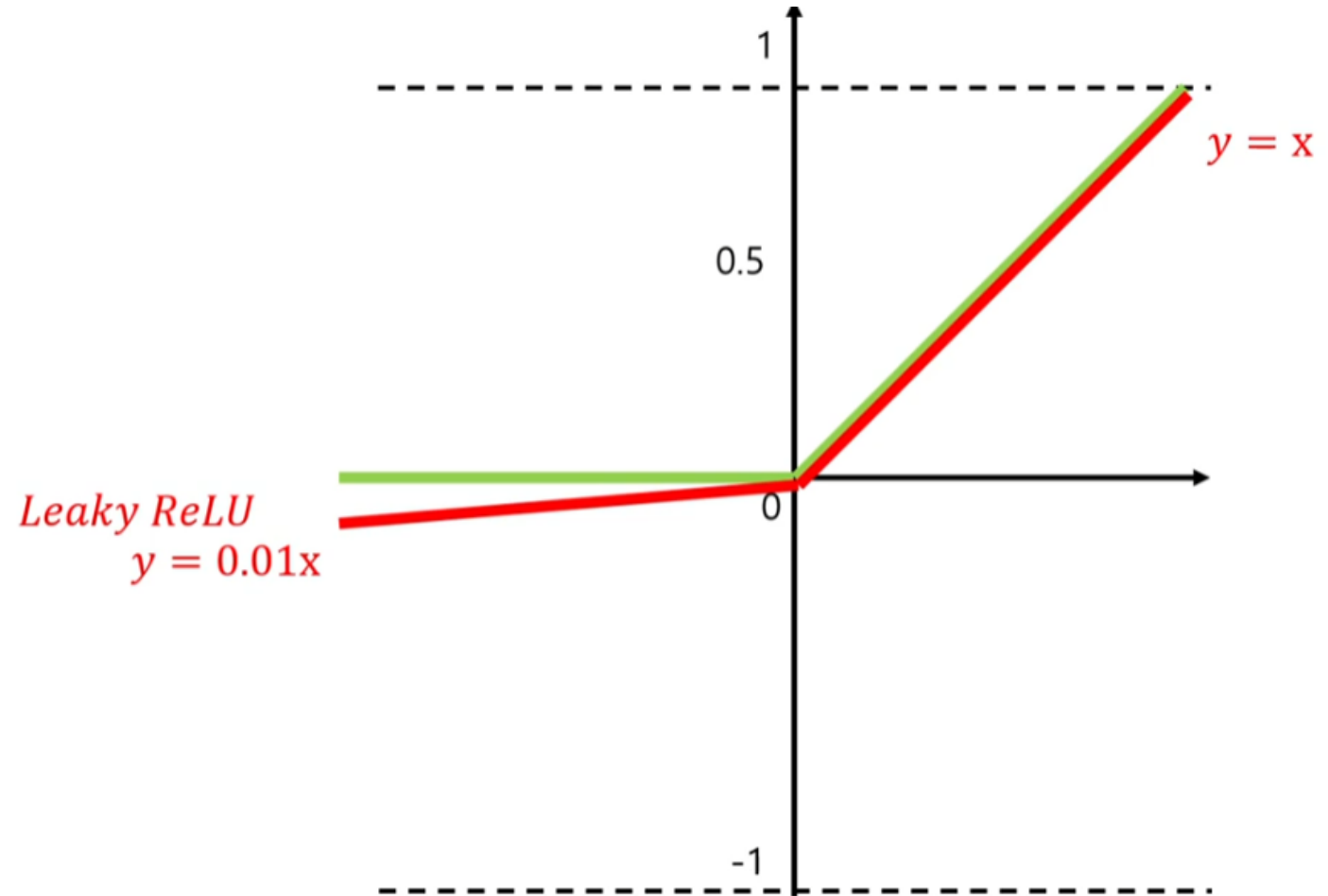
"Dying ReLU 현상 발생"



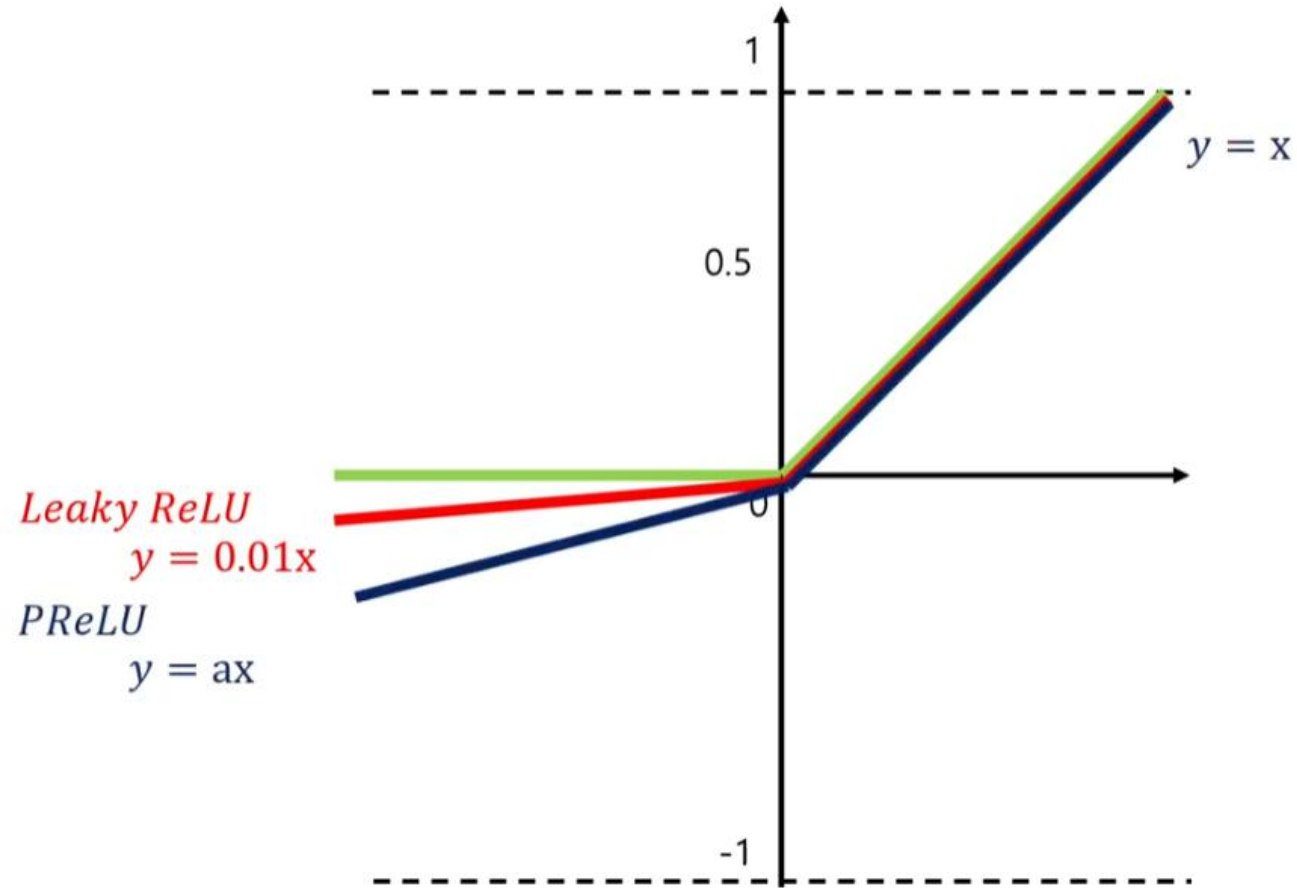
변형 ReLU 'Leaky ReLU'

'음수 입력값에 작은 slope를 줌'

'입력값 = 음수, 학습가능'



PReLU '음수 기울기 = a'
'a값 자유롭게 설정'.....학습 효율 높임



3. Regularization(정규화)

- 목적: 과적합(overfitting) 줄임
- 과적합이란?

“AI가 문제와 정답을 통째로 외워버리는 학습 때문에 처음 보는 문제에 제대로 대응하지 못하는 비효율적인 현상”

“너무 똑똑한 AI는 외우기만 해서, 처음 보는 문제를 못 풀어요.”

“그래서 AI가 외우지 말고, 진짜 이해하게 도와주는 훈련규칙이

Regularization이에요!

『훈련규칙= '너무 복잡하게 외우지 말고, 꼭 필요한 것만 기억하라'』

L1 정규화

- 모델에서 중요한 특성만을 선택하고, 나머지는 제거하여 모델을 "가볍게" 만든다.

<특성 선택>

- 가중치의 절대값의 합에 대한 패널티를 손실함수에 추가
- 학습과정에서 정규화 항 때문에 가중치의 값이 0으로 수렴
- 가중치가 0이 되는 특성을 모델 출력에 영향을 주지 않으므로, 이 특성들은 제거
- 가중치가 0이 아닌 특성들만 선택하여 모델에서 사용

L1 정규화 예시 “스팸 메일 분류기”

1. 훈련규칙= ‘너무 복잡하게 외우지 말고, 꼭 필요한 것만 기억하라’
2. 문제점: “이 메일에 들어 있는 단어들 중에,
‘무료’, ‘광고’, ‘지금 클릭’,
혹은 그냥 ‘안녕하세요’, ‘회의’, ‘첨부파일’ 같은 평범한 단어도 전부!”

“처음에는 수천 개의 단어를 다 중요한 것처럼 취급한다. 그러나 대부분의 단어는 스팸과 관련이 없다”

“그래서, L1 정규화 등장한다. 즉 관련없는 단어는 과감하게 무시하라”

“즉, 신경망 관점에서 보면 관련없는 단어는 L1 정규화를 통해 가중치를 0으로 만들고, 단어를 삭제한다.”

모델이 단순해지고, 과적합 방지되며, 성능이 향상된다.

“L1 정규화 목표함수: 수식의 값 최소화”

- 모델 성능을 향상시키기 위해서는 “중요하지 않은 특성의 가중치는 0으로 수렴하도록 유도”
- 핵심 특성은 그대로 유지

$$L(\theta) = \text{Original Loss Function}(y, f(X; \theta)) + \lambda \sum_{i=1}^n |\theta_i|$$

여기서:

“L1 정규화는 과적합 방지를 위해
손실함수에 부과되는 추가적인 비용 ”

- $L(\theta)$ 는 정규화된 손실 함수입니다.
- $\text{Original Loss Function}(y, f(X; \theta))$ 는 원래의 손실 함수로, 예측값 $f(X; \theta)$ 와 실제값 y 의 차이를 측정합니다.
- λ 는 정규화 강도를 조절하는 하이퍼파라미터이며, 이 값은 사용자가 정하며 패널티의 강도를 결정합니다.
- $\sum_{i=1}^n |\theta_i|$ 는 모델 가중치 θ 의 절대값의 합으로, L1 노름(norm)이라고 합니다.
- n 은 모델 가중치의 개수입니다.

“예측에 도움이 되지 않는 특성은 그 가중치를 아예 0으로 만들고 싶다.”

L1 정규화 수식 설명

1. L1 정규화 수식은 머신러닝 모델의 목표함수(Objective Function)로, 이 수식의 값을 최소화하는 것이 학습의 목표입니다.
2. 손실함수에 가중치들의 절댓값 합(정규화 항)을 더함으로써, 중요하지 않은 특성의 가중치는 0으로 수렴하게 유도합니다.
3. 하지만 중요한 특성은 모델 성능에 영향을 주기 때문에, 수식의 최소화 과정에서도 그 가중치는 유지되며, 결과적으로 '필요한 것만 남기는 모델'이 만들어집니다.

L2 정규화

$$L(\theta) = \text{Original Loss Function} + \lambda \sum_{i=1}^n \theta_i^2$$

- 오류에 관여한 모든 가중치 제공의 합을 손실함수에 추가하여 각 가중치가 너무 큰 값을 가지지 않도록 한다.
- 모든 가중치가 비교적 작은 값을 유지하도록 하여, 모델이 훈련 데이터의 작은 특징이나, 노이즈에 지나치게 민감하게 반응하지 않도록 한다.

“특정 Feature에 올인하지 마라 ”

스팸 메일 분류로 설명하면,

1. L1 정규화: "중요 단어만 선택하라"

"무료", "클릭", "지금 구매" 같은 **핵심 단어만** 골라서
나머지 평범한 단어는 **완전히 제거함**

2. L2 정규화: "특정 단어에 올인하지 마라"

AI가 "무료"라는 단어에만 올인하지 않게 하고,
"할인", "이벤트", "쿠폰", "광고" 등도 **적당히** 고려하도록 만듦

즉, 무료=스팸이라 분류하면, 무료 주차권 첨부도 무료라는 단어 때문에 스팸으로 분류한다.

L2 정규화 수식 설명

1. L2 정규화 수식은 손실함수에 가중치의 제곱합을 더한 형태의 목표 함수이며, 이 수식의 값을 최소화하는 것이 학습의 핵심 목표입니다.
2. 수식 값을 최소화 하기 위해서는 가중치가 클수록 더 큰 벌점이 부과해야 한다.
3. 가중치가 클수록 더 큰 벌점을 주면, 모델은 전체 가중치를 고르게 작게 유지하려는 방향으로 학습이 진행됩니다.
4. 즉, 여러 특성을 조금씩 고르게 참조하고, 소수의 특성에 몰인하지 않게 된다.

4. Regularization rate(정규화 계수, 비율)

- 정규화할 때 어느 정도 값을 수정해 줄까? 사용하는 값
- L2 정규화 예: 가중치가 큰 단어에 벌점을 더 많이 준다

“정규화 계수 λ 는 “큰 가중치를 가진 단어일수록, 얼마나 더 강하게 벌점을 줄 것인지 정하는 조절기”

$$L(\theta) = \text{Original Loss Function} + \boxed{\lambda} \sum_{i=1}^n \theta_i^2$$

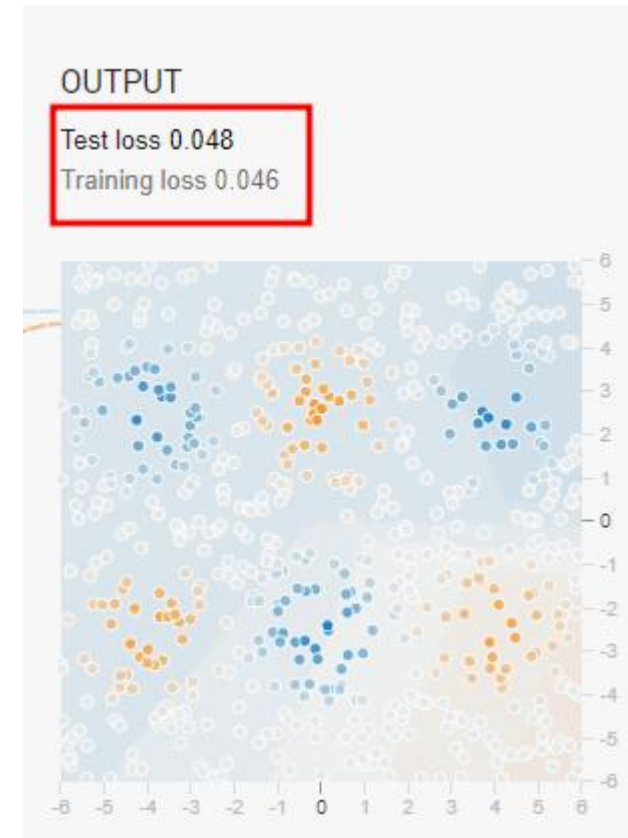
5. Problem type

- 분류(Classification): 데이터를 주황색과 파란색의 데이터로 분류하도록 모델을 학습시키는 것
예) 고양이인가? 강아지인가?
- 회귀(Regression) : 연속된 데이터의 값을 예측하도록 모델을 학습시키는 것
예) 주가예측, 날씨예측

**“입력데이터 값 + 노이즈 + 배치사이즈 +
특징 + 뉴런(히든레이어)”**

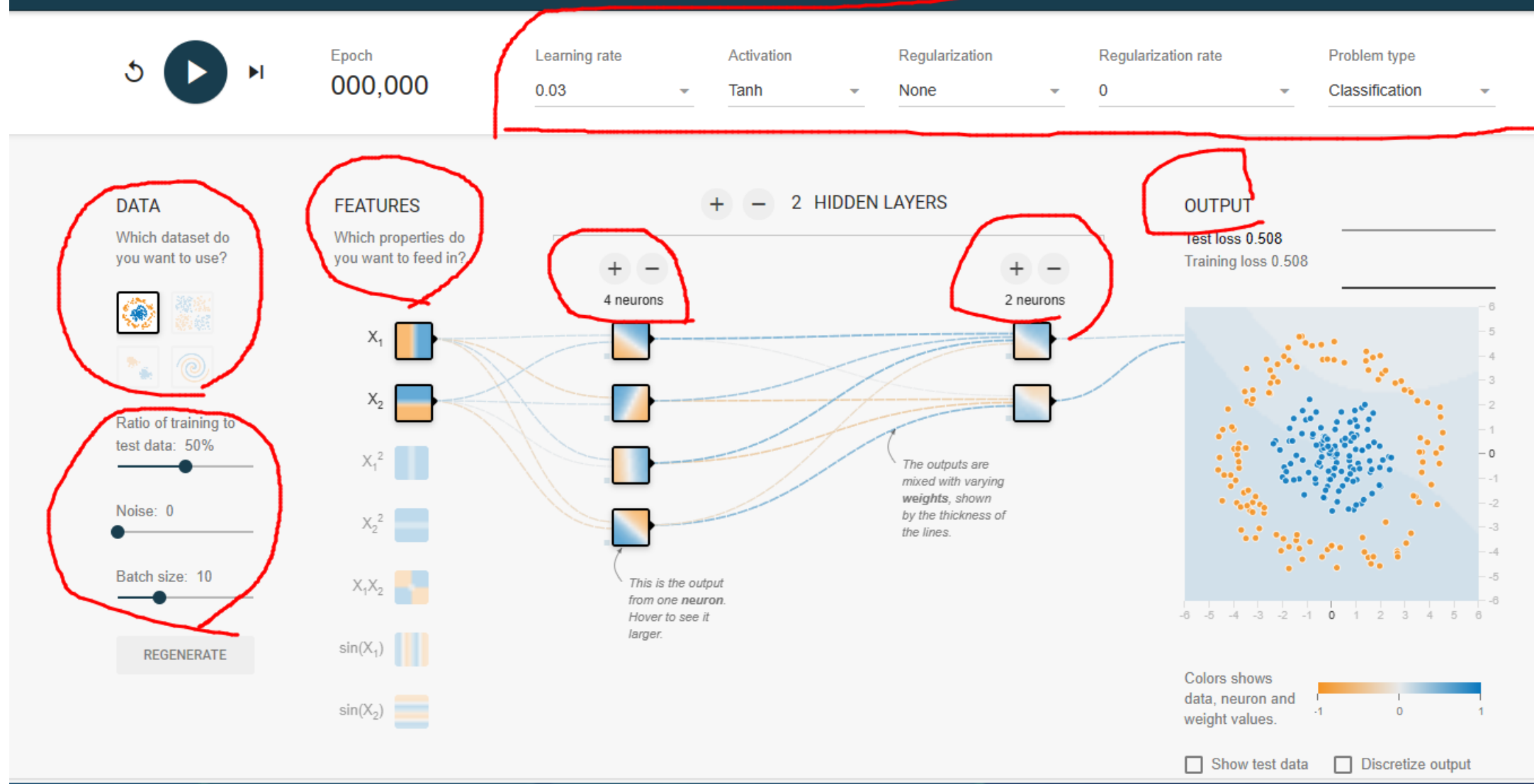
“마음 놓고 마음대로 조정해보세요 ”

“출력의 Test Loss, Training loss값이
0에 가까울 수록 학습이 잘 일어난 것
입니다.”

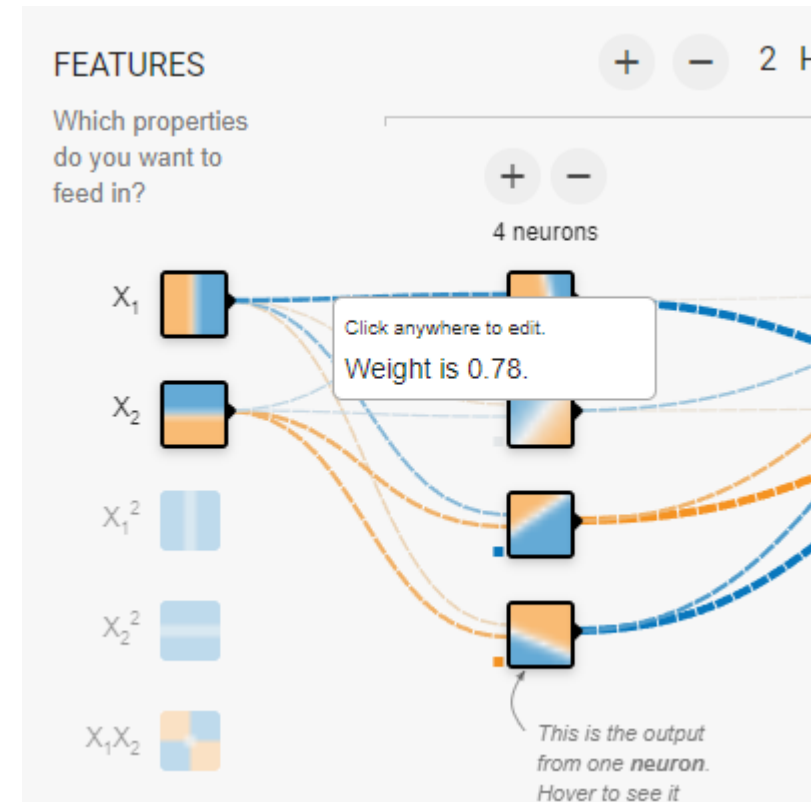
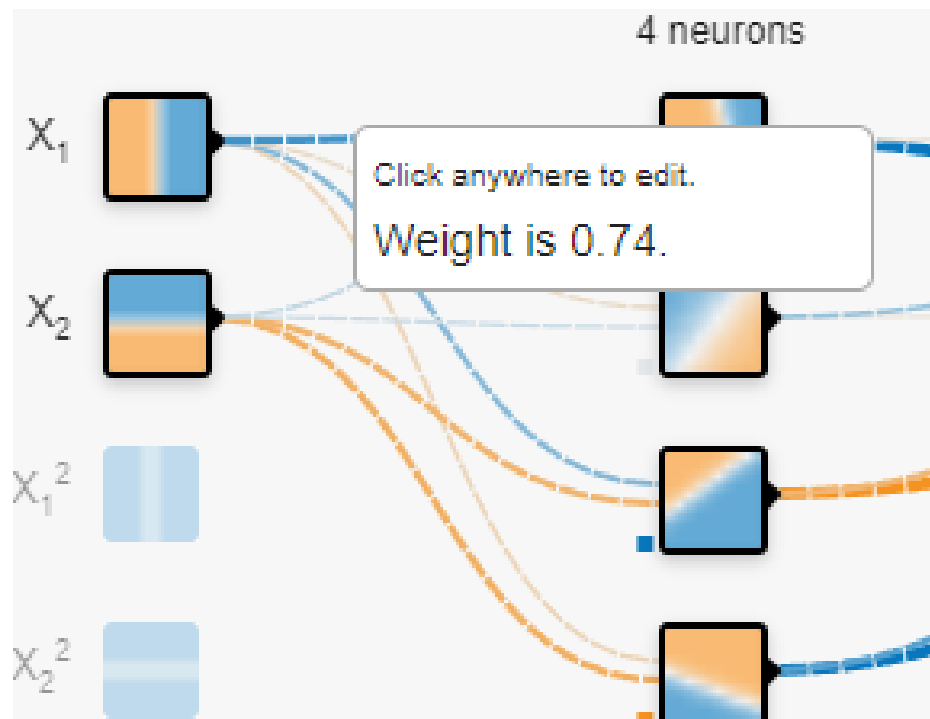


Tinker With a **Neural Network** Right Here in Your Browser.

Don't Worry, You Can't Break It. We Promise.



Weight(파라미터) 값이 변함 "학습의 과정"





Epoch
000,000

Learning rate

0.3

Activation

ReLU

Regularization

None

Regularization rate

0

Problem type

Regression

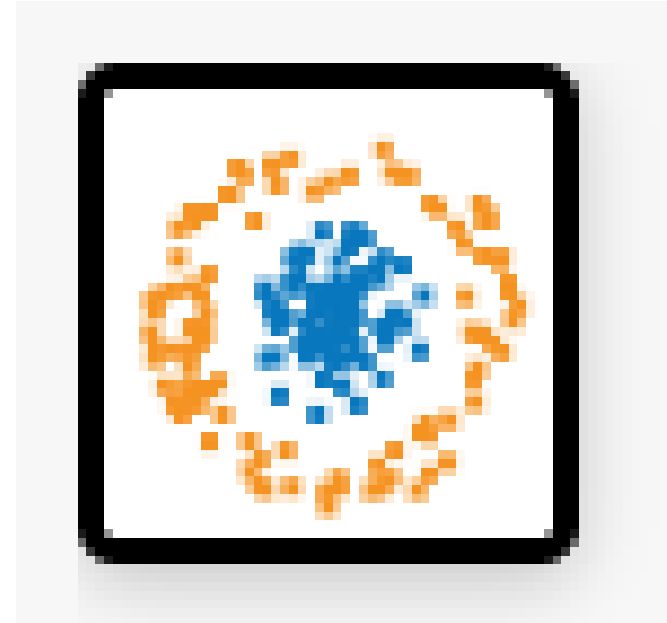
“옵션을 잘 설정해야

학습이 잘 이루어진다”

“마음 놓고 마음대로 조정해보세요 ”

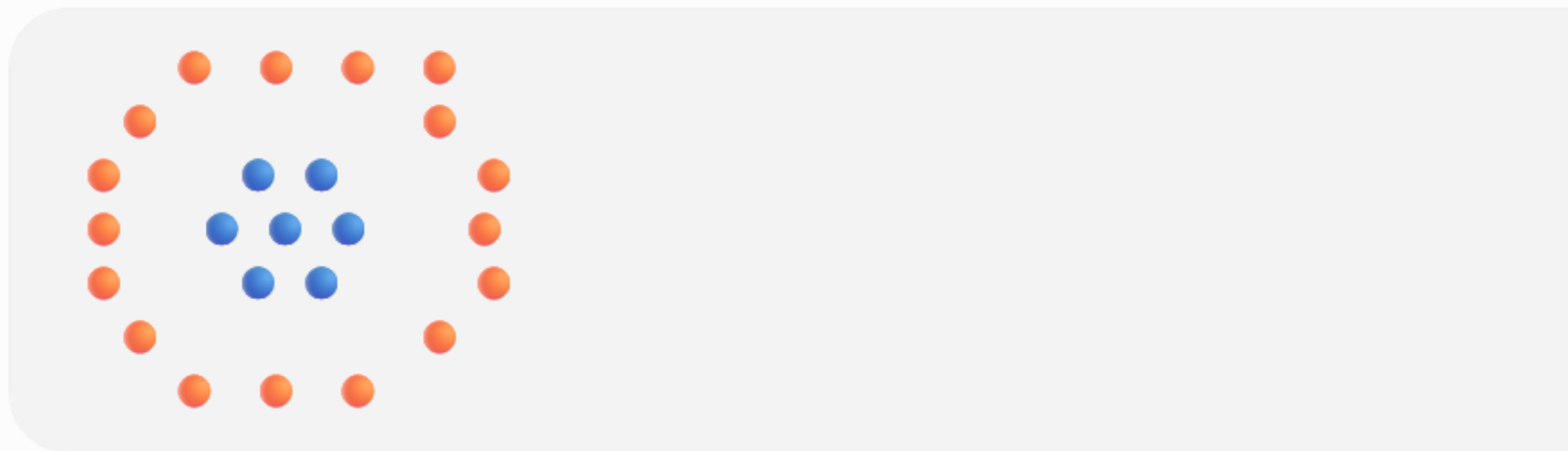
데이터 셋 1(중앙집중형 VS 원형분포)

1. 이름: Circular / Donut
2. 비선형 데이터: 직선 하나로 분류할 수 없음
3. 딥러닝 모델
 - Deep Neural Network (DNN) with Nonlinear Activations(비선형 활성화 함수+DNN)
 - Convolutional Neural Networks (CNNs)



1. Circle 데이터셋이란?

Circle 데이터셋은 데이터가 원의 안쪽과 바깥쪽으로 나누어진 형태입니다.



핵심: 직선 하나로는 ●과 ●을 구분할 수 없는
비선형 분류(Non-linear Classification) 문제

2. 실생활 예 — 학교 통학거리

학교를 중심으로 학생들의 집 위치를 나타낸다고 생각해 봅시다.

구분	의미
 중심	학교
 파란색	학교에서 2km 이내 거주 학생
 주황색	학교에서 2km 초과 거주 학생
X_1	동쪽 ↔ 서쪽 위치
X_2	남쪽 ↔ 북쪽 위치

학교에서 일정 거리 이내인지 판단하기 때문에 **원 모양의 경계**가 만들어집니다.

판단 원리

$X_1^2 + X_2^2 < \text{기준값}$ →  원 안쪽

$X_1^2 + X_2^2 > \text{기준값}$ →  원 바깥쪽

따라서 단순한 **직선**으로는 구분하기 어렵고, 신경망이 여러 뉴런을 조합하여 **곡선 형태의 결정경계 (Decision Boundary)**를 학습합니다.

실제 상황

학생들의 집은 학교를 중심으로 여러 방향에 흩어져 있습니다.

학생 A: 학교에서 동쪽으로 1km, 북쪽으로 0.5km

 **AI:** "학교와 가깝습니다. →  안쪽 집단"

학생 B: 학교에서 서쪽으로 2km, 남쪽으로 2km

 **AI:** "학교에서 멀리 떨어져 있습니다. →  바깥쪽 집단"

AI는 단순히 **동쪽으로 얼마나 떨어졌는지(x_1)**만 보는 것이 아닙니다.

x_1 과 x_2 를 함께 고려하여 학교 중심에서의 거리를 판단해야 합니다.

3. 다양한 실생활 사례

분야

 원 안쪽

 원 바깥쪽

 통학

학교 2km 이내

2km 초과

 배달

배달 가능 지역

배달 불가 지역

 Wi-Fi

신호가 강한 지역

신호가 약한 지역

 드론

통신 가능 범위

통신 범위 밖

 농업

스프링클러 급수 범위

급수 범위 밖

 상권

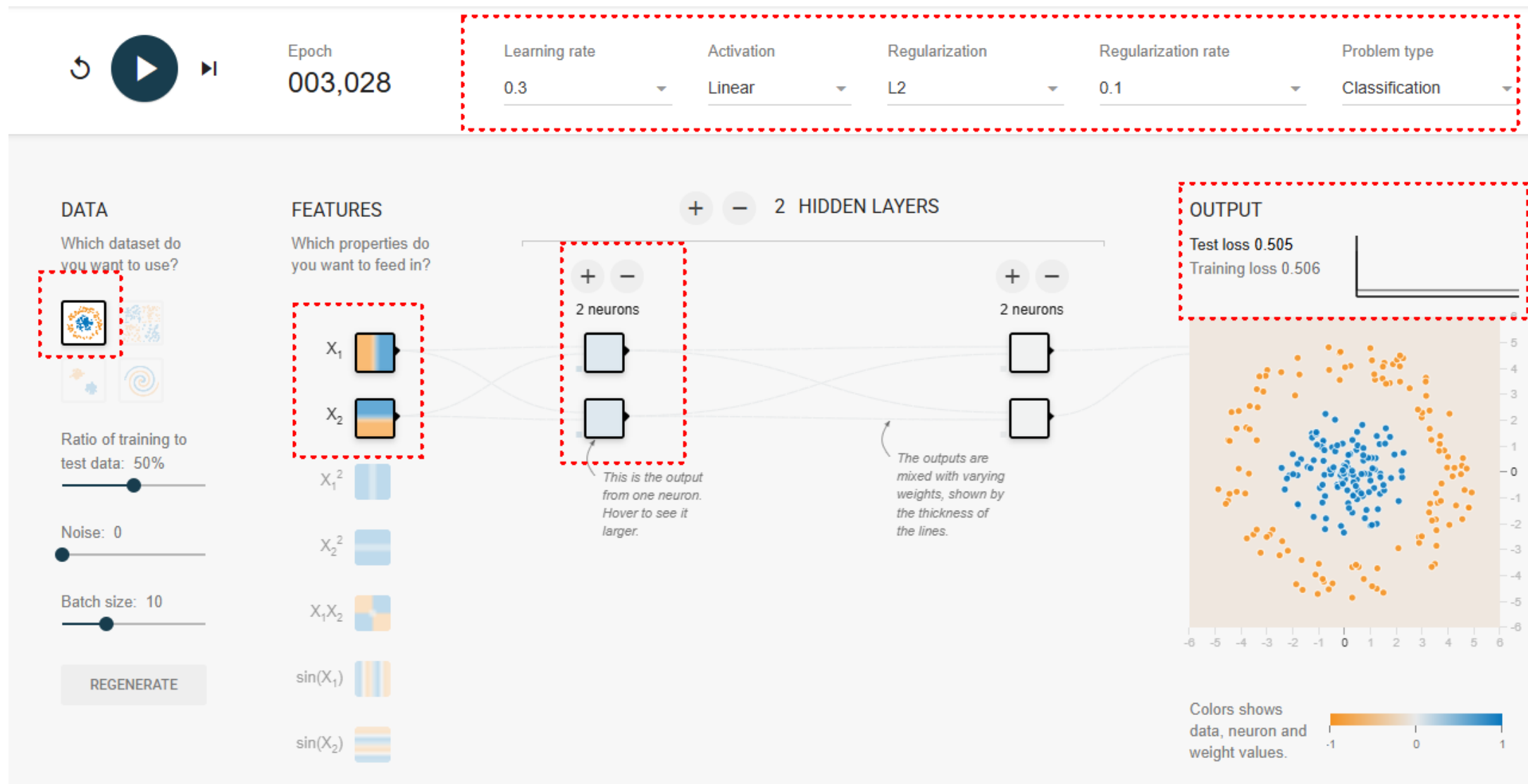
매장 핵심 상권

외곽 상권

한 줄 정리

Circle 데이터셋 = 중심에서의 거리에 따라 안쪽과 바깥쪽을 구분하는 대표적인 비선형 분류 문제

성능이 나쁘게 나오는 경우



Circle 데이터

1) 성능이 나쁘게 나오는 경우

- 데이터셋: Circle
- **Features:** x_1, x_2 (기본 좌표만)
- **Hidden Layers:** 1층, 뉴런 2개
- **Activation:** Linear
- **Learning Rate:** 0.3 (너무 큼)
- **Regularization:** L2, $\lambda=0.1$ (너무 강함)
- **Problem type:** Classification

👉 결과

- 결정 경계(Decision boundary)가 직선 비슷하게 나옴.
- 원형 패턴을 잡지 못함 → 학습 성능 낮음 (Train/Test 정확도 모두 낮음).
- Loss 값도 진동하거나 발산할 수 있음.

! 이유

1. **Linear** 활성화 → 직선적 경계만 학습 가능 → 원형 구조 인식 불가.
2. 너무 높은 학습률 → 최적점에 도달하지 못하고 발산.
3. 규제 λ 가 크다 → 모델이 단순해져서 패턴 학습 불가.
4. 은닉층/뉴런 부족 → 원형 같은 비선형 패턴 표현이 불가능.
5. **Features** 제한적 → x_1, x_2 만 사용하면 원형 구조 포착 어려움.

(모범)TensorFlow Playground 설정

설정 항목	추천값	설명
데이터 셋	Circle	가운데/바깥 분리된 도넛 구조
Features	x1 , x2 , x1^2 , x2^2 , x1 * x2	원형 경계 인식 가능
Hidden Layers	3~4개	최소 3개 이상 필요
Nodes per Layer	각 층마다 6~10개	비선형 표현력 확보
Activation	tanh 또는 ReLU	tanh 이 더 안정적으로 학습됨
Learning rate	0.03 ~ 0.1	중간값으로 시작, 점진적 조정
Regularization	L2 0.001 정도	과적합 방지용
Noise	기본값 또는 약간 추가	일반화에 도움



Epoch
000,757

Learning rate
0.03

Activation
Tanh

Regularization
L2

Regularization rate
0

Problem type
Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 10

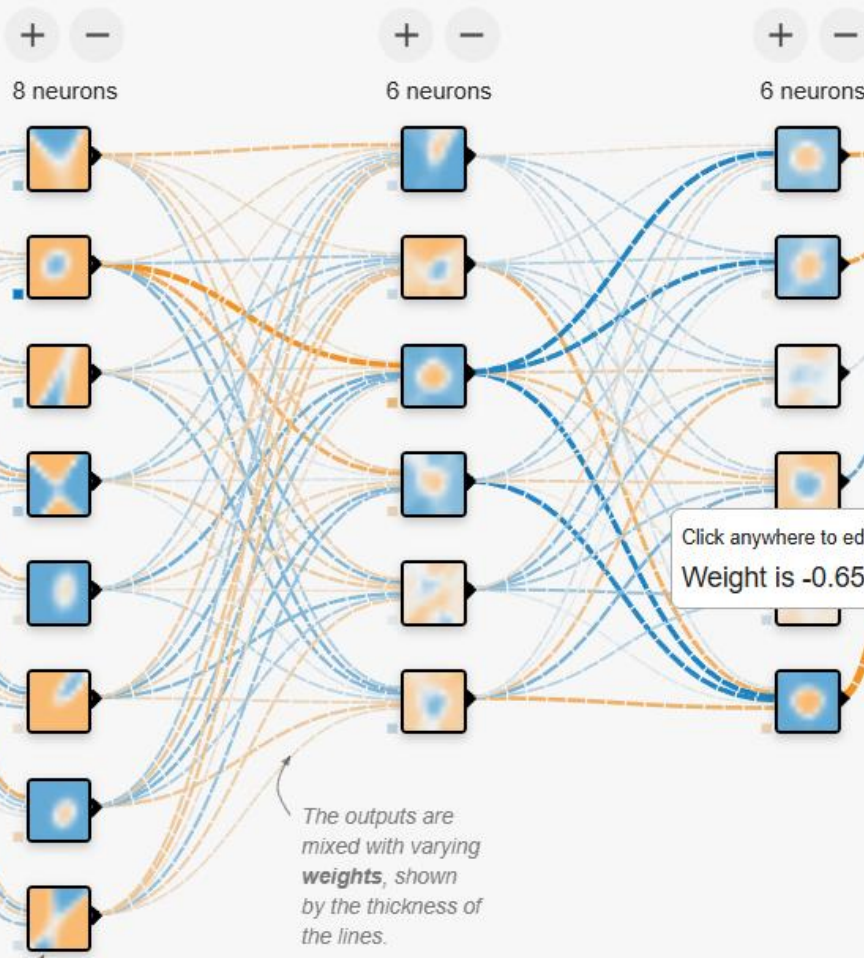
REGENERATE

FEATURES

Which properties do you want to feed in?

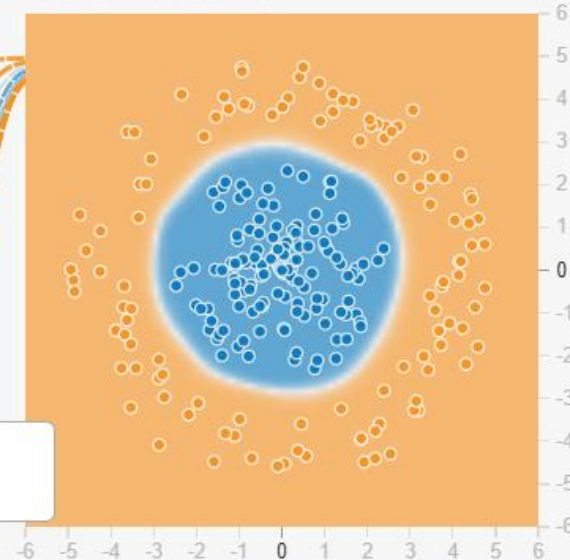
X_1
 X_2
 X_1^2
 X_2^2
 X_1X_2
 $\sin(X_1)$
 $\sin(X_2)$

+ - 3 HIDDEN LAYERS



OUTPUT

Test loss 0.000
Training loss 0.000



Colors shows data, neuron and weight values.

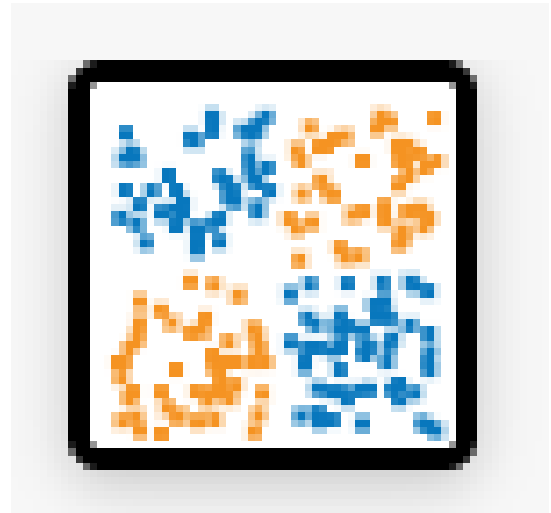


☐ Show test data ☐ Discretize output

The outputs are mixed with varying **weights**, shown by the thickness of the lines.

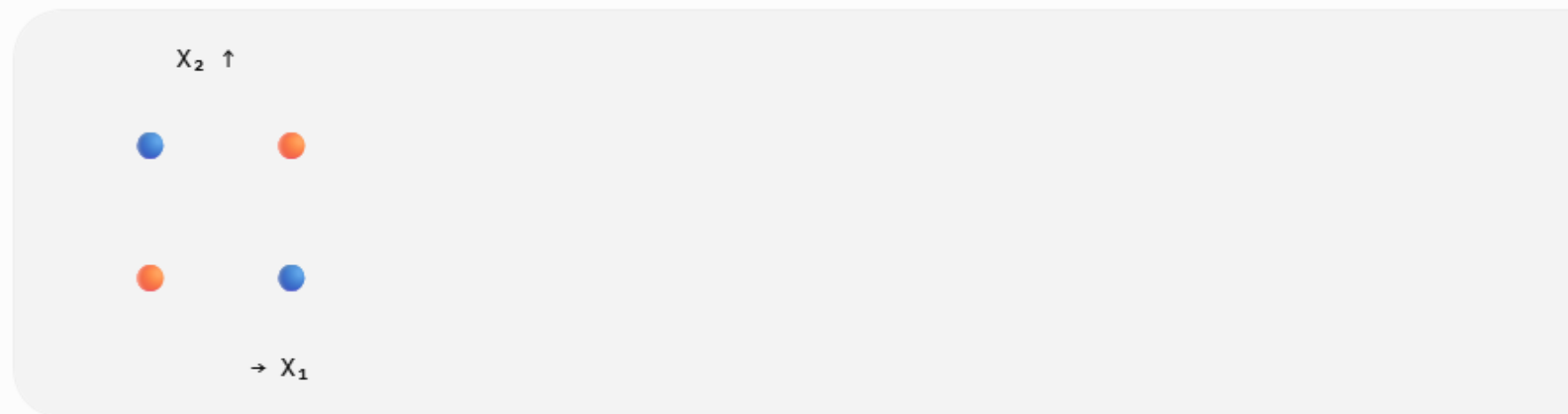
데이터 셋 2(격자모양 분포)

1. 이름: Grid or XOR-type pattern
2. 비선형 데이터: 직선 하나로 분류할 수 없음
3. 딥러닝 모델
 - 다층 퍼셉트론(MLP, DNN)
 - **SVM with RBF Kernel** (서포트 벡터 머신 + 방사형 기저 함수)



1. XOR 데이터셋이란?

XOR(Exclusive OR, 배타적 논리합)은 두 조건이 서로 다를 때만 1, 같으면 0이 되는 데이터입니다.



즉, 같은 색이 대각선 방향으로 배치됩니다.

X_1	X_2	결과
0	0	0
0	1	1
1	0	1
1	1	0

★ 핵심: 직선 하나로 ●과 ●을 나눌 수 없는 대표적인 비선형 분류 문제입니다.

✕◯ XOR 데이터셋 — 실생활 사례

XOR은 아주 쉽게 말하면 “두 조건이 서로 다르면 한 그룹, 서로 같으면 다른 그룹”으로 분류하는 문제입니다.

💡 가장 쉬운 사례 — 두 개의 전등 스위치

복도나 계단에서 스위치 2개로 전등 하나를 켜고 끄는 경우를 생각하면 쉽습니다.

스위치 A	스위치 B	전등
OFF	OFF	● 꺼짐
OFF	ON	💡 켜짐
ON	OFF	💡 켜짐
ON	ON	● 꺼짐

즉,

두 스위치의 상태가 서로 다르면 → 전등 ON

두 스위치의 상태가 같으면 → 전등 OFF

이것이 전형적인 XOR 관계입니다.

🤖 AI 분류 문제로 바꾸면

- X_1 = 스위치 A의 상태
- X_2 = 스위치 B의 상태
- ● = 전등 ON
- ● = 전등 OFF

그래프로 나타내면:



같은 색깔이 **대각선 방향**에 있기 때문에 직선 하나로는 ● 과 ● 을 구분할 수 없습니다.

💡 3. 실생활에서 생각할 수 있는 사례

사례	X_1	X_2	XOR의 의미
💡 조명 스위치	스위치 A	스위치 B	서로 다른 상태일 때 점등
📄 출입 시스템	카드 인증	비밀번호 인증	둘 중 하나만 인증
💻 시스템 상태	서버 A	서버 B	한쪽만 작동하는 상태 탐지
📶 센서	센서 A	센서 B	두 센서 판단이 서로 다름
🤖 AI 판단	모델 A	모델 B	두 모델의 예측이 불일치

특히 두 개의 스위치로 하나의 전등을 제어하는 상황을 생각하면 XOR을 이해하기 쉽습니다.

실제 상황

제품 하나가 생산라인을 지나갑니다.

센서 A: "정상입니다."

센서 B: "이상입니다."



AI:

"두 센서의 판단이 서로 다릅니다. 이 제품은 **재검사가 필요합니다.**"

다음 제품에서는

센서 A: "정상입니다."

센서 B: "정상입니다."



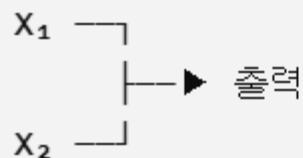
AI:

"두 센서의 판단이 같습니다. **정상 처리합니다.**"

4. 신경망에서 XOR이 중요한 이유

XOR은 퍼셉트론(Perceptron)의 한계를 설명할 때 매우 유명한 문제입니다.

단층 신경망



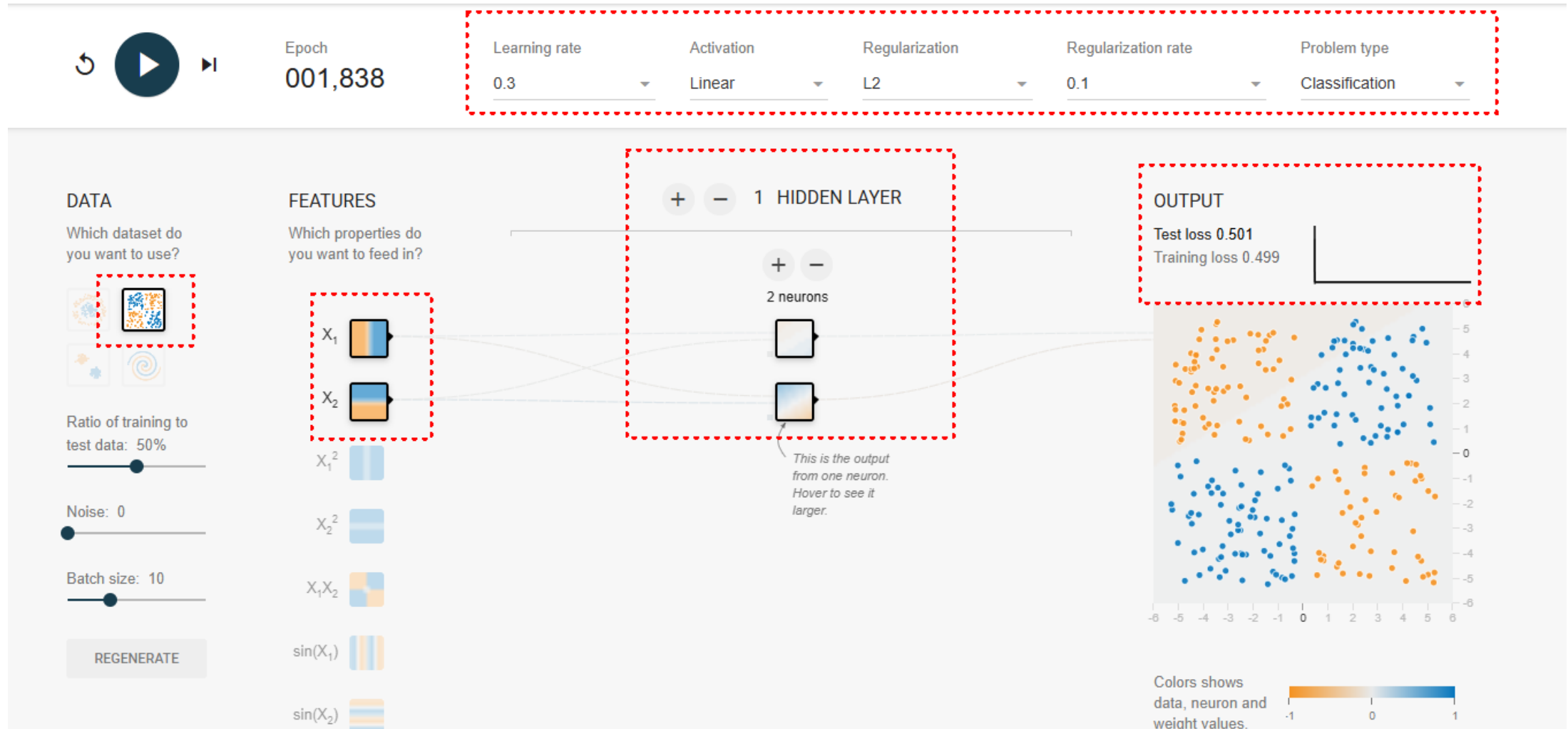
❌ 직선 하나의 결정경계만 만들 수 있으므로 XOR을 제대로 분류할 수 없습니다.

은닉층이 있는 신경망



✅ 여러 뉴런이 서로 다른 경계를 만들고 이를 결합하면 XOR도 분류할 수 있습니다.

(XOR)성능이 나쁘게 나오는 경우



XOR 데이터

1) 성능이 나쁘게 나오는 경우

- **Problem type:** Classification
- **Features:** x_1, x_2 (기본 특징만 사용)
- **Hidden Layers:** 1층, 뉴런 2개
- **Activation:** Linear (비선형성 없음)
- **Learning Rate:** 0.3 (너무 큼)
- **Regularization:** L2, $\lambda=0.1$ (규제 강함)

! 이유

1. **Linear** 활성화: XOR 문제는 본질적으로 비선형 → 선형 경계로는 절대 분리 불가.
2. 은닉층과 뉴런 부족: 최소 2층 이상 + 충분한 뉴런 필요.
3. 높은 학습률: 가중치가 발산하거나 안정적 수렴 불가.
4. 규제가 강함: 모델 복잡성을 억제해 더 단순해짐.

👉 결과

- 결정 경계가 직선 두 개 수준 → XOR 데이터는 대각선 구분만 가능해서 완전히 실패.
- Train/Test 정확도 모두 낮음 (~50% 수준).
- Loss 값도 안정적으로 줄지 않고 진동할 수 있음.

(모범) TensorFlow Playground 설정

- **Problem type:** Classification
- **Features:** $x_1, x_2, x_1 \times x_2, x_1^2, x_2^2$ (파생 특징 활용)
- **Hidden Layers:** 2층, 각 층 4~8 뉴런
- **Activation:** Tanh 또는 ReLU (비선형성 보장)
- **Learning Rate:** 0.03 (적절)
- **Regularization:** 없음 ($\lambda=0$)

👉 결과

- 네트워크가 네 개의 사분면을 올바르게 구분.
- Train/Test 정확도 95% 이상 달성.
- Loss가 안정적으로 줄어듦.



Epoch

001,286

Learning rate

0.03

Activation

Tanh

Regularization

L2

Regularization rate

0

Problem type

Classification

DATA

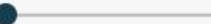
Which dataset do you want to use?



Ratio of training to test data: 50%



Noise: 0



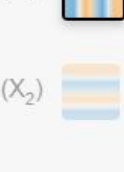
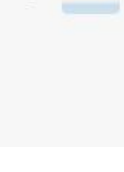
Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?

 X_1  X_2  X_1^2  X_2^2  $X_1 X_2$  $\sin(X_1)$  $\sin(X_2)$ 

+ - 3 HIDDEN LAYERS

+ -

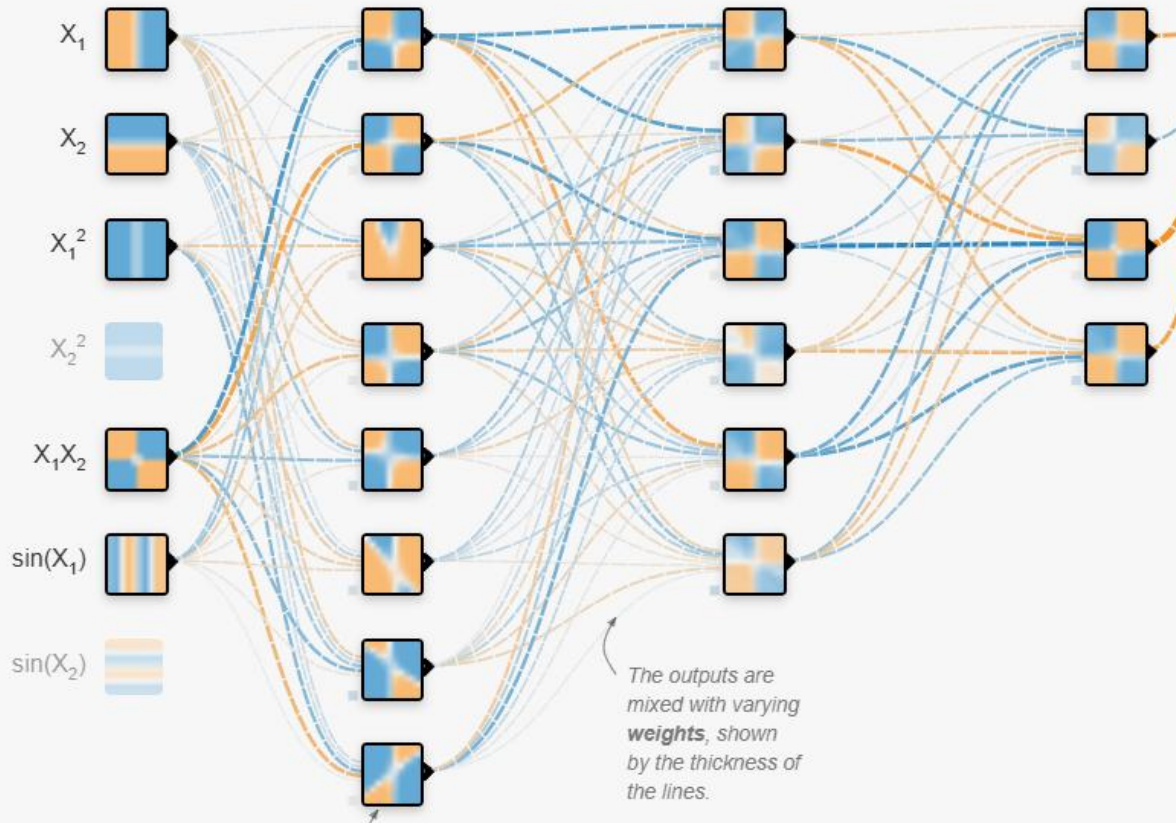
8 neurons

+ -

6 neurons

+ -

4 neurons

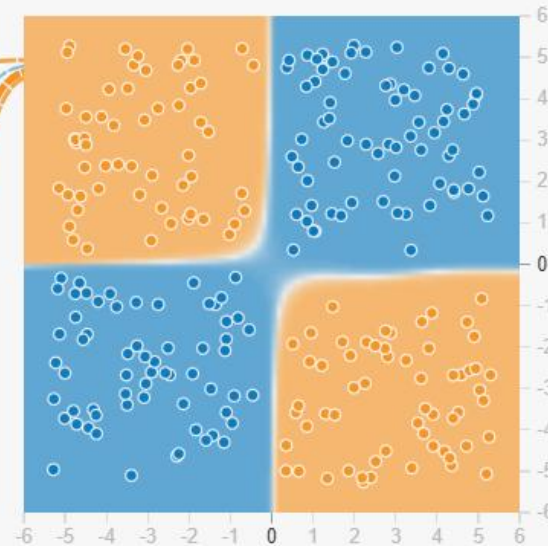


The outputs are mixed with varying **weights**, shown by the thickness of the lines.

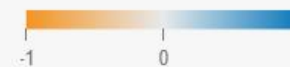
OUTPUT

Test loss 0.007

Training loss 0.000



Colors shows data, neuron and weight values.

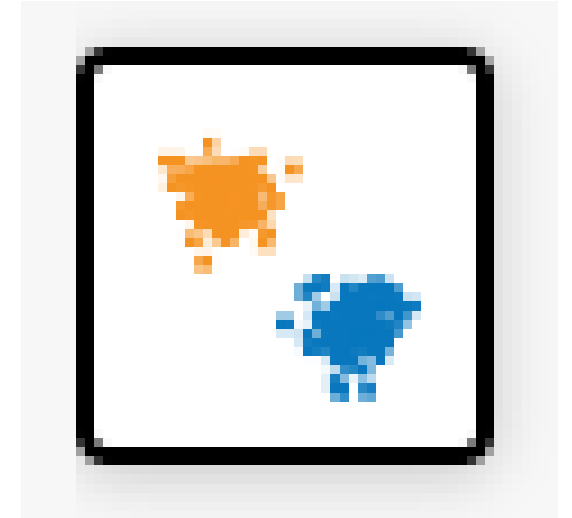
☐ Show test data☐ Discretize output

정리(XOR 데이터)

구분	나쁜 설정 (성능 낮음)	좋은 설정 (성능 높음)	이유
Learning Rate	0.3 (너무 큼)	0.03 (적절)	큰 값은 발산, 적절한 값은 안정적 학습
Activation	Linear	Tanh / ReLU	XOR는 비선형 → Linear는 학습 불가
Regularization	L2, $\lambda=0.1$	없음 ($\lambda=0$)	강한 규제는 복잡한 경계 학습 불가
Problem Type	Classification	Classification	문제 유형 동일
Features	x_1, x_2, x_1, x_2 만 사용	$x_1, x_2, x_1x_2, x_1^2, x_2^2, x_1x_2^2, x_1^2x_2$	파생 특징이 복잡한 패턴 학습에 도움
Hidden Layers	1층, 뉴런 2개	2층, 각 4~8뉴런	깊이와 너비가 있어야 XOR 해결 가능

데이터 셋 3(군집형)

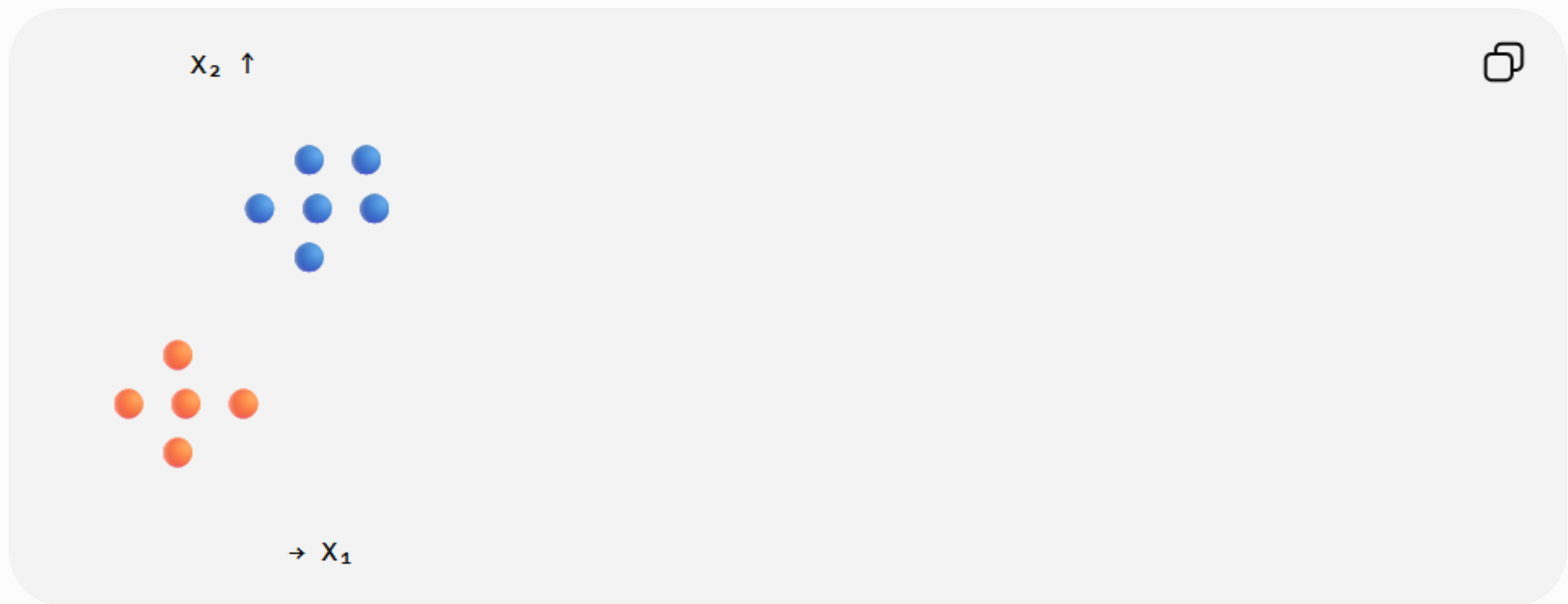
1. 이름: Gaussian blobs or Linearly Separable
2. 선형 분리 가능
 - 고양이 vs 강아지처럼 특징이 뚜렷이 다른 두 클래스
3. 딥러닝 모델
 - Logistic regression, Perceptron, Linear SVM 등 간단한 모델로도 분류 가능



Gaussian 데이터셋 = 비슷한 특징을 가진 데이터들이 두 개의 덩어리로 모여 있고, AI가 두 집단을 구분하도록 학습하는 대표적인 분류 데이터셋입니다.

1. Gaussian 데이터셋이란?

Gaussian(가우시안) 데이터셋은 데이터가 두 개의 덩어리(군집)처럼 모여 있는 형태입니다.



각 집단의 중심에는 데이터가 많이 모이고, 중심에서 멀어질수록 데이터가 적어지는 **정규 분포(Gaussian Distribution)** 형태를 갖습니다.

★ **핵심:** 두 집단의 특징이 서로 달라서 비교적 직선으로 쉽게 분류할 수 있는 데이터셋



2. 쉬운 실생활 예 — 학생의 학습 특성

학생을 두 가지 특징으로 표현한다고 가정해 보겠습니다.

- X_1 = 수학 점수
- X_2 = 과학 점수

과학점수 ↑



학업성취 높음



학업성취 낮음

→ 수학점수

AI가 해야 할 일

학생들의 수학·과학 점수를 보고

- 학업성취가 높은 집단과
- 학업성취가 낮은 집단

을 구분하는 것입니다.

두 집단 사이에 직선 하나를 그어도 비교적 잘 구분할 수 있습니다.

실제 상황

학생 A

수학 92점 / 과학 88점



AI:

"수학과 과학 점수가 모두 높은 학생들이 모여 있는 영역에 있습니다."

→ ● 높은 성취 집단

학생 B

수학 55점 / 과학 60점



AI:

"수학과 과학 점수가 상대적으로 낮은 학생들이 모여 있는 영역에 있습니다."

→ ● 기초학습 지원 집단

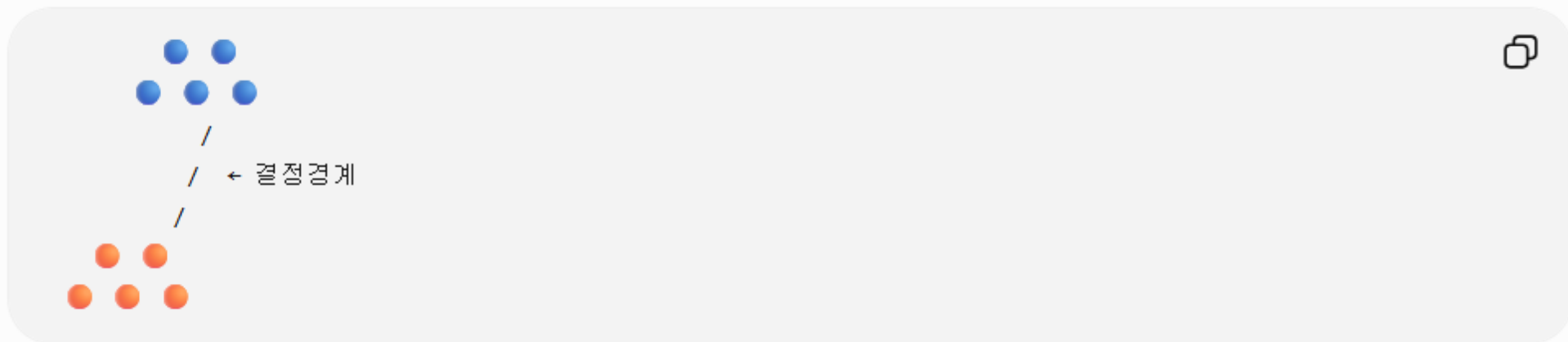
● ● Gaussian 데이터셋: AI가 수학·과학 점수처럼 두 가지 특징을 분석하여 비슷한 특성을 가진 두 집단을 구분하는 문제

3. 실생활 사례

분야	X_1	X_2	분류
 교육	수학 점수	과학 점수	성취 집단 분류
 건강	혈압	혈당	정상군 ↔ 위험군
 금융	소득	신용점수	저위험 ↔ 고위험
 제조	온도	진동	정상 ↔ 이상
 농업	온도	습도	생육 양호 ↔ 불량
 AI	특징 1	특징 2	클래스 A ↔ 클래스 B

🧠 4. 신경망에서는 왜 쉬울까?

Gaussian 데이터셋은 보통 두 집단이 떨어져 있기 때문에 **선형 결정경계(Linear Decision Boundary)**만으로도 상당 부분 분류할 수 있습니다.



따라서 TensorFlow Playground에서 신경망의 기본적인 분류 원리를 처음 실험하기 좋은 데이터셋입니다.

★ Circle · XOR · Gaussian 비교

데이터셋	모양	분류 난이도	특징	📄
● Gaussian	두 덩어리	★ 쉬움	직선으로 구분 가능	
○ Circle	안쪽·바깥쪽	★★ 중간	원형 경계 필요	
✕ XOR	대각선 배치	★★★ 어려움	여러 경계 조합 필요	

Gaussian 데이터 셋

1) 성능이 나쁘게 나오는 경우

- **Problem type:** Classification
- **Features:** x_1, x_2 (기본 특징만 사용)
- **Hidden Layers:** 1층, 뉴런 1개
- **Activation:** Linear
- **Learning Rate:** 0.3 (너무 큼)
- **Regularization:** L2, $\lambda=0.1$ (규제 강함)

👉 결과

- 결정 경계가 지나치게 단순(직선 하나) → 두 개의 점 구름(클러스터)을 제대로 나누지 못함.
- Train 정확도 60~70% 수준, Test 정확도도 낮음.
- 학습률이 너무 커서 손실(Loss) 값이 안정적으로 줄지 않고 흔들릴 수 있음.

! 이유

1. **Linear 활성화 + 은닉층 부족** → 직선형 경계밖에 표현 못함.
2. **너무 큰 학습률** → 최적점 근처에서 발산.
3. **규제 λ 가 크다** → 모델 복잡도가 억제되어 유연한 결정 경계 학습 불가.
4. **특징 부족** → 단순히 x_1, x_2 만 사용하면 비선형적 구분 어려움.

(모범)TensorFlow Playground 설정

- **Problem type:** Classification
- **Features:** x_1, x_2, x_1^2, x_2^2 (파생 특징 추가)
- **Hidden Layers:** 2층, 각 4~6 뉴런
- **Activation:** Tanh 또는 ReLU
- **Learning Rate:** 0.03 (안정적)
- **Regularization:** 없음 ($\lambda=0$)

👉 결과

- 두 개의 가우시안 클러스터를 곡선형 결정 경계로 매끄럽게 분리.
- Train/Test 정확도 95% 이상.
- Loss가 안정적으로 빠르게 감소.



Epoch
002,003

Learning rate

0.3

Activation

Linear

Regularization

L2

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?

X_1

X_2

X_1^2

X_2^2

$X_1 X_2$

$\sin(X_1)$

$\sin(X_2)$

+ - 2 HIDDEN LAYERS

+ -

6 neurons

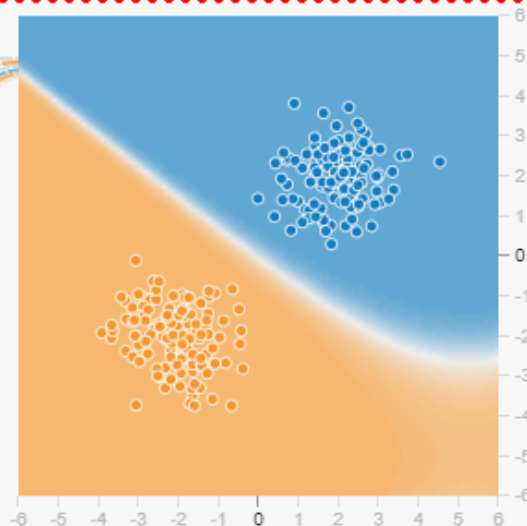
+ -

6 neurons

OUTPUT

Test loss 0.000

Training loss 0.000



Colors shows data, neuron and weight values.

This is the output from one neuron. Hover to see it

The outputs are mixed with varying weights, shown by the thickness of

데이터 셋 4(나선형)

1. 이름: Spiral
2. 가장 어려운 분류 문제
 - 선형분류 불가능
 - 여러층의 깊은 신경망+비선형성 필요
 - 학습이 오래 걸림
 - 복잡한 시계열 패턴이나 손글씨 식별에서 유사구조 나타남
3. 딥러닝 모델
 - 다층 퍼셉트론(MLP / DNN)
 - Radial Basis Function Network (RBF Network)



1. 나선형 데이터셋이란?

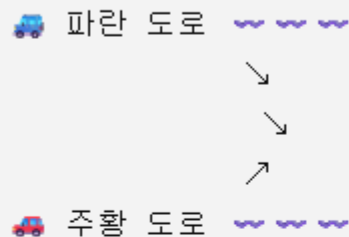
Spiral 데이터셋은 서로 다른 두 종류의 데이터가 나선 모양으로 서로 얹혀 있는 데이터입니다.



★ **핵심:** 단순한 직선이나 원 하나로는 분류하기 어려운
복잡한 비선형 분류(**Non-linear Classification**) 문제입니다.

2. 실생활 예 — 산길에서 차량 구분하기

두 개의 구불구불한 산길이 있다고 생각하면 가장 쉽습니다.



두 도로가 산을 따라 여러 번 휘어져 있고 서로 가까이 지나갑니다.

자동차의 위치 (x_1 , x_2)만 보고 AI가

- 파란 도로의 자동차
- 주황 도로의 자동차

를 구분해야 한다고 생각해 보세요.

직선 하나를 그어서는 절대 두 도로를 구분할 수 없습니다.

AI가 도로의 굽은 모양을 따라가며 복잡한 경계를 만들어야 합니다.

“구불구불하게 서로 얹혀 있는 두 산길에서 자동차의 위치를 보고, 파란 도로를 주행하는 자동차인지 주황 도로를 주행하는 자동차인지 분류하는 문제”



4가지 데이터셋 비교

데이터셋	모양	핵심 특징	난이도
● Gaussian	두 덩어리	직선으로 구분 가능	★
○ Circle	안쪽 ↔ 바깥쪽	원형 경계 필요	★★
✕ XOR	대각선 배치	여러 경계 필요	★★★
🌀 Spiral	나선형으로 얹힘	매우 복잡한 곡선 경계 필요	★★★★

★ 한 줄 정리

Spiral 데이터셋 = 두 집단이 복잡하게 얹혀 있어, 신경망이 여러 층과 뉴런을 이용해 복잡한 비선형 결정경계를 학습하는 대표적인 데이터셋

쉽게 말하면 **Gaussian → Circle → XOR → Spiral**로 갈수록 신경망이 더 복잡한 패턴을 학습해야 한다고 설명할 수 있습니다.

TensorFlow Playground 설정

항목	추천 설정	설명
Features	x1 , x2 , x1*x2 , x1^2 , x2^2	곡선/대칭 구조 인식용
Hidden Layers	최소 4개 이상	깊은 신경망 필요
Nodes per Layer	8~12개 추천	복잡한 경계 생성
Activation	tanh 	곡선형 경계에 적합 (ReLU 보다 더 매끄러움)
Learning Rate	0.03~0.1	수렴 속도 조절
Regularization	L2 0.001	과적합 방지
Epochs	1000 이상	복잡한 구조 학습 필요



Epoch
001,203

Learning rate

0.03

Activation

Tanh

Regularization

L2

Regularization rate

0

Problem type

Classification

DATA

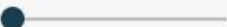
Which dataset do you want to use?



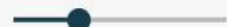
Ratio of training to test data: 50%



Noise: 0



Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?

X_1



X_2



X_1^2



X_2^2



$X_1 X_2$



$\sin(X_1)$



$\sin(X_2)$



+ - 4 HIDDEN LAYERS

+ -

8 neurons

+ -

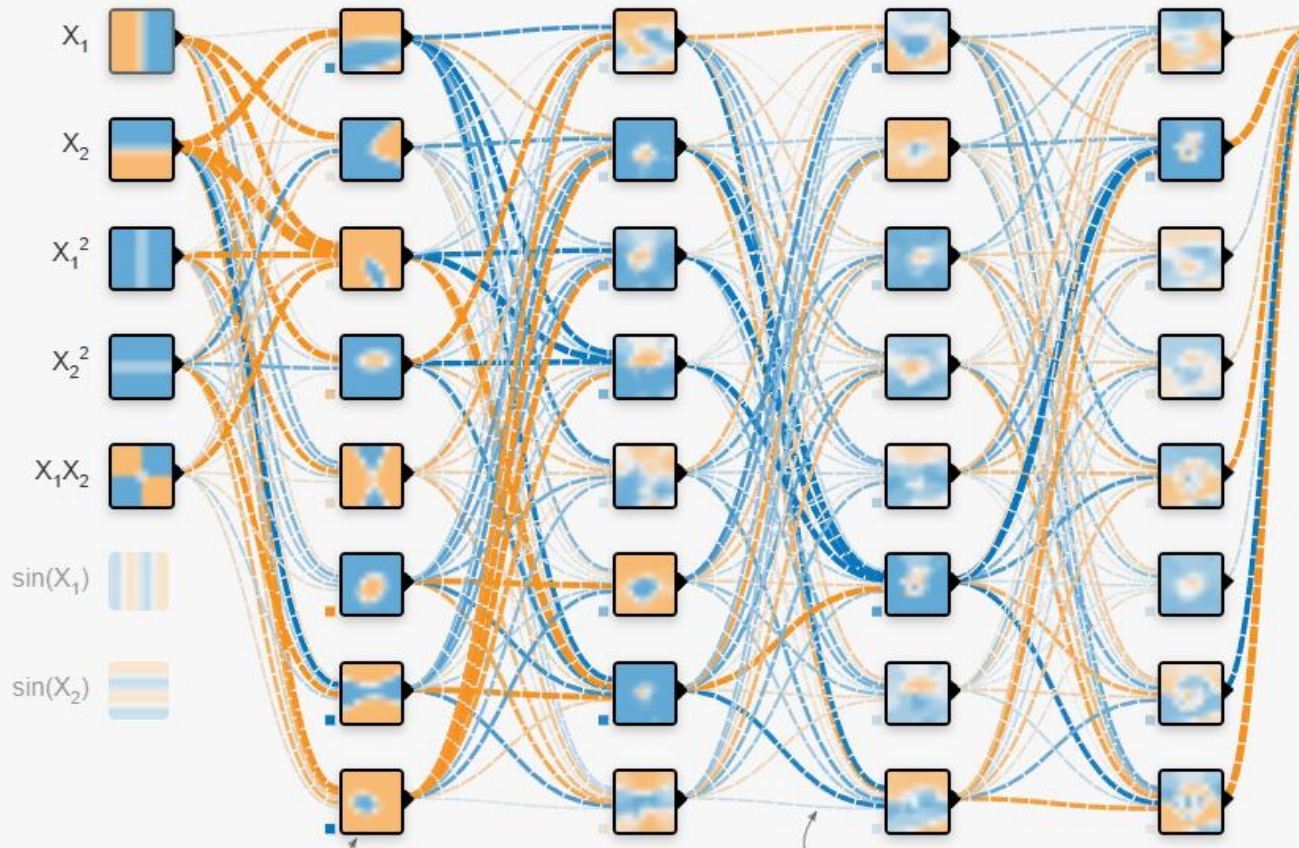
8 neurons

+ -

8 neurons

+ -

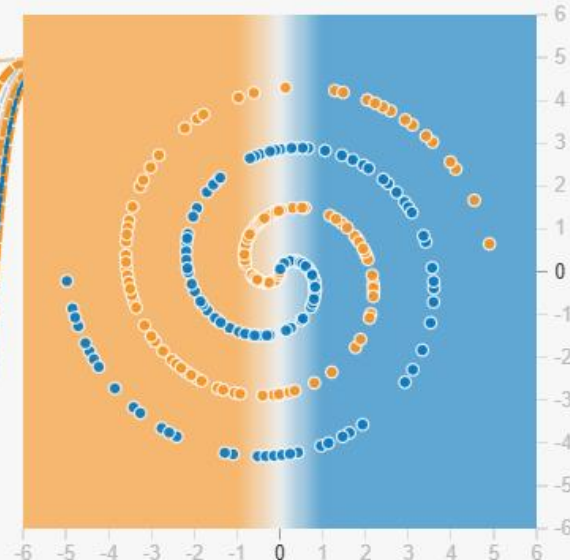
8 neurons



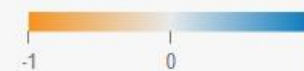
OUTPUT

Test loss 0.010

Training loss 0.000



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output

4가지의 데이터가 포함된 사례



자율주행 AI가 도로 상황을 판단하는 사례

자율주행 자동차는 카메라·레이더·GPS 등의 데이터를 받아 주행해도 되는지, 멈춰야 하는지, 어떤 차량을 주의해야 하는지 판단합니다.

데이터셋	쉽게 말하면	 실제 상황	 AI 역할
 Gaussian	두 덩어리	빠른 차량 ↔ 느린 차량	비슷한 차량끼리 분류
 Circle	안 ↔ 밖	내 차 주변 20m	위험범위 안/밖 판단
 XOR	조건의 조합	신호등과 차량 움직임이 불일치	특이 상황 발견
 Spiral	복잡하게 얹힘	복잡한 교차로의 차량 움 직임	복잡한 주행패턴 판단

① Gaussian — "어느 집단인가?"

고속도로에서 주변 차량들의 **속도와 가속도**를 측정합니다.

  느리게 움직이는 차량들이 한쪽에 모이고
  빠르게 움직이는 차량들이 다른 쪽에 모입니다.

 AI:

"이 차량은 빠르게 움직이는 차량 집단에 가깝다."

 **Gaussian = 비슷한 특징을 가진 데이터끼리 모인 집단을 구분**

② Circle — "위험범위 안인가?"

내 자동차를 중심으로 20m를 안전거리라고 생각해 봅시다.



다른 차량이 20m 안으로 들어옵니다.

🤖 AI:

"차량이 위험범위 안으로 들어왔다. 속도를 줄이자."

👉 Circle = 중심을 기준으로 안과 밖을 구분

❌ ③ XOR — "두 정보가 서로 맞는가?"

AI가 두 가지 정보를 확인합니다.

신호등: ● 진행 가능

앞 차량: ● 정지

두 정보가 서로 다릅니다.

🤖 AI:

"신호는 녹색인데 앞 차량이 멈춰 있다.
평상시와 다른 상황이다. 주의하자."

반대로

● 신호 + 🚗 차량 진행

→ 서로 일치

● 신호 + 🚗 차량 정지

→ 서로 일치

④ Spiral — "움직임이 너무 복잡하다"

복잡한 로터리를 생각해 보겠습니다.



자동차들이 직선으로만 움직이지 않습니다.

- 🚗 좌회전
- 🚗 우회전
- 🚗 차선 변경
- 🚗 회전
- 🚶 보행자 이동

🤖 AI:

"단순한 직선 경계로 판단할 수 없다. 여러 특징을 함께 분석해서 차량들의 이동 패턴을 구분하자."

👉 **Spiral** = 복잡하게 얹혀 있는 패턴을 구분

여러 움직임이 복잡하게 얹혀 있습니다.

- **Gaussian** → "어느 무리에 속하지?"
- **Circle** → "위험범위 안이야, 밖이야?"
- ✕ **XOR** → "두 조건의 조합이 어떨지?"
- 🌀 **Spiral** → "복잡하게 얽힌 패턴이 무엇이지?"

결국 **AI의 역할**은 같습니다.

👁️ 데이터를 보고 → 🧠 패턴을 학습하고 → 🚗 새로운 상황을 분류·판단하는
것

TensorFlow Playground에서는 바로 이 과정을 단순한 2차원 데이터로 눈으로 확인하는
것입니다.

다시 실습해 봅시다~

<https://playground.tensorflow.org/>